Projet Mathématiques pour le Big Data — Optimisation, Pseudo-inverse, ACP, AFC

ESGI 5A BD

2026-03-30

Contents

1 Exercice 1 — Moindres carrés **4**

1.1 Données et transformation logarithmique 4

1.2 Droite d'ajustement par les moindres carrés 4

1.3 Ajustement exponentiel 5

1.4 Graphiques 5

2 Exercice 2 — Optimisation sans contrainte **9**

2.1 Exercice 2.2.1 : $f(x, y) = e^{xy} - e^x + 2$ 9

2.1.1 Recherche des points critiques 9

2.1.2 Matrice hessienne et nature du point 9

2.1.3 Tracé sur $[0, 2]^2$ 10

2.2 Exercice 2.2.2 : $g(x, y, z) = (x + z^2)e^{x(y^2+z^2+1)}$ 12

2.3 Exercice 2.2.3 : $h(x, y, z) = \ln(xyz) - \ln(x)\ln(y)\ln(z)$ sur $]0, +\infty[^3$ 14

3 Exercice 3 — Optimisation sous contraintes **17**

3.1 Exercice 3.2.1 : $f(x, y) = xy$ sous $g(x, y) = x - y + 6 = 0$ 17

3.1.1 Tracé 19

3.2 Exercice 3.2.2 : $f(x, y, z) = x^2 + y^2 + z^2$ sous $g_1 = x + y + z - 1 = 0$ et $g_2 = z = 0$ 20

3.3 Exercice 3.2.3 : $f = x^2 + y^2 + z^2$ sous $g_1 = x^2 + y^2 + 2z - 6 = 0$ et $g_2 = x - y - z = 0$ 22

3.4 Exercice 3.2.4 : $f(x, y) = x^2y$ sous $2x^2 + y^2 = 3$ 24

3.4.1 Tracé 25

3.5 Exercice 3.2.5 : $f(x, y) = \frac{x^2y^2}{x^2+y^2} + 2x^2 + 2y^2 + 4xy + 5x + 5y + 20$ sous $2x^2 + 2y^2 + 4xy - 8 = 3$ 27

3.5.1 Tracé 28

4 Exercice 4 — Décomposition en valeurs singulières et pseudo-inverse **30**
4.1 Exercice 4.2 : SVD — Cas 1 . . . 30
4.1.1 SVD de la matrice A . . . 30
4.1.2 SVD de la matrice B . . . 32
4.2 Exercice 4.3 : SVD — Cas 2 (Pseudo-inverse et résolution) . . . 34
4.2.1 Pseudo-inverse de A . . . 34
4.2.2 Pseudo-inverse de B . . . 35
4.2.3 Résolution de Ax = b et Bx = b . . . 37

5 Exercice 5 — ACP (Analyse en Composantes Principales) **39**
5.1 Partie 1 : Première approche . . . 39
5.1.1 Chargement des données . . . 39
5.1.2 Question 1 : Matrice de corrélation . . . 39
5.1.3 Question 2 : Variables corrélées / décorrélées . . . 41
5.1.4 Question 3 : Graphiques de couples . . . 42
5.2 Partie 2 : ACP centrée-réduite . . . 45
5.2.1 Question 4 : Résultats de l'ACP . . . 45
5.2.2 Question 5 : Répartition de l'inertie . . . 47
5.2.3 Question 6 : Aliments déterminant les axes . . . 48
5.2.4 Question 7 : Pays déterminant les axes . . . 50
5.2.5 Question 8 : Interprétation des axes . . . 52
5.2.6 Question 9 : Cinq pays mal représentés . . . 53
5.2.7 Question 10 : Cas du Portugal . . . 53
5.3 Partie 3 : Variable supplémentaire EST . . . 54
5.3.1 Question 11 : Données suffisantes ? . . . 54
5.3.2 Question 12 : Conditions et interprétation . . . 55

6 Exercice 6 — Test du Chi-deux **56**

7 Exercice 7 — AFC : Cas 1 **58**
7.1 Partie 1 : Introduction . . . 58
7.1.1 Question 1 : Pourcentages . . . 58
7.1.2 Question 2 : Sondés et proportions . . . 58
7.2 Partie 2 : AFC . . . 59
7.2.1 Question 3 : Pourquoi 7 valeurs propres ? . . . 60
7.2.2 Question 4 : Modalités définissant les axes . . . 60
7.2.3 Question 5 : Modalités mal représentées . . . 61
7.2.4 Question 6 : Proximités . . . 62

8 Exercice 8 — AFC : Élection présidentielle 2007 **64**

8.1 Test du Chi-deux 65

8.2 AFC 65

1 Exercice 1 — Moindres carrés

1.1 Données et transformation logarithmique

On dispose des données suivantes sur la revente d'un article :

```
t <- c(0, 1, 2, 3, 4, 5, 6)
Xt <- c(90, 73.8, 60, 49.5, 40.5, 33, 27)
n <- length(t)

# Affichage du tableau
knitr::kable(data.frame(t = t, Xt = Xt),
             col.names = c("t (années)", "Xt (centaines €)"),
             caption = "Prix de revente en fonction du temps")
```

Table 1: Prix de revente en fonction du temps

t (années)	Xt (centaines €)
0	90.0
1	73.8
2	60.0
3	49.5
4	40.5
5	33.0
6	27.0

```
# Transformation logarithmique
Zt <- log(Xt)
cat("Valeurs de Zt = ln(Xt) :\n")
```

```
## Valeurs de Zt = ln(Xt) :
```

```
print(round(Zt, 4))
```

```
## [1] 4.4998 4.3014 4.0943 3.9020 3.7013 3.4965 3.2958
```

1.2 Droite d'ajustement par les moindres carrés

On cherche $Z_t = at + b$ par la méthode des moindres carrés (sans fonction prédéfinie).

```
# Moyennes
t_bar <- sum(t) / n
Z_bar <- sum(Zt) / n

# Pente a = Cov(t,Z) / Var(t)
cov_tZ <- sum((t - t_bar) * (Zt - Z_bar))
var_t  <- sum((t - t_bar)^2)
a <- cov_tZ / var_t
```

```r
# Ordonnée à l'origine
b <- Z_bar - a * t_bar

cat("Pente a =", round(a, 6), "\n")
```

```
## Pente a = -0.200524
```

```r
cat("Ordonnée à l'origine b =", round(b, 6), "\n")
```

```
## Ordonnée à l'origine b = 4.500304
```

```r
cat("Droite d'ajustement : Z(t) =", round(a, 4), "× t +", round(b, 4), "\n")
```

```
## Droite d'ajustement : Z(t) = -0.2005 × t + 4.5003
```

1.3 Ajustement exponentiel

Puisque $Z_t = \ln(X_t) = at + b$, on en déduit $X_t = e^{at+b} = e^b \cdot e^{at}$.

```r
A_coeff <- exp(b)
cat("Modèle exponentiel : X(t) =", round(A_coeff, 4), "× exp(", round(a, 4), "× t)\n")
```

```
## Modèle exponentiel : X(t) = 90.0445 × exp( -0.2005 × t)
```

```r
# Estimation à t = 10
t10 <- 10
X10 <- exp(a * t10 + b)
cat("Estimation à t = 10 ans :", round(X10, 2), "centaines d'euros\n")
```

```
## Estimation à t = 10 ans : 12.12 centaines d'euros
```

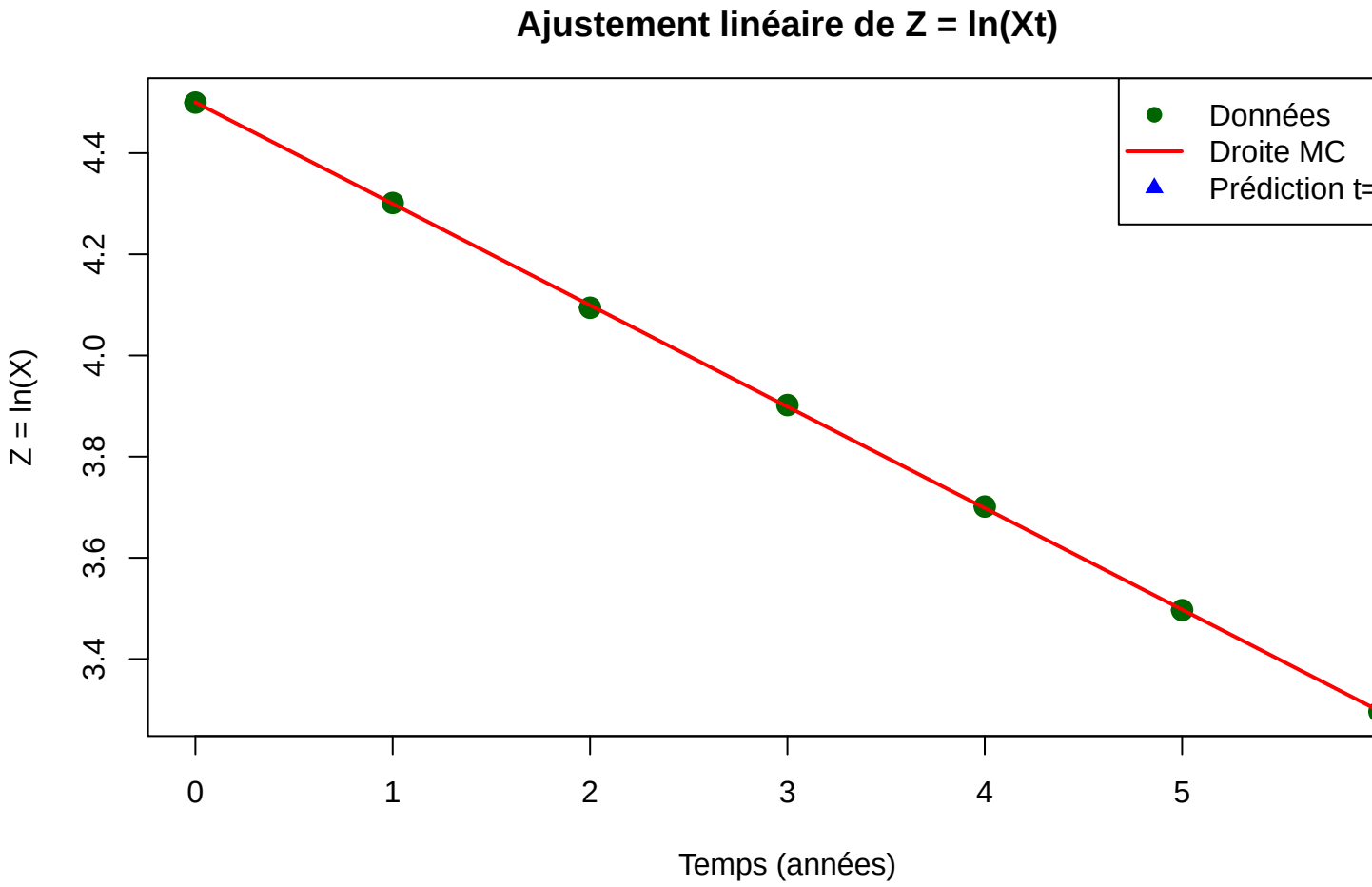
```r
cat("Soit environ", round(X10 * 100, 0), "euros\n")
```

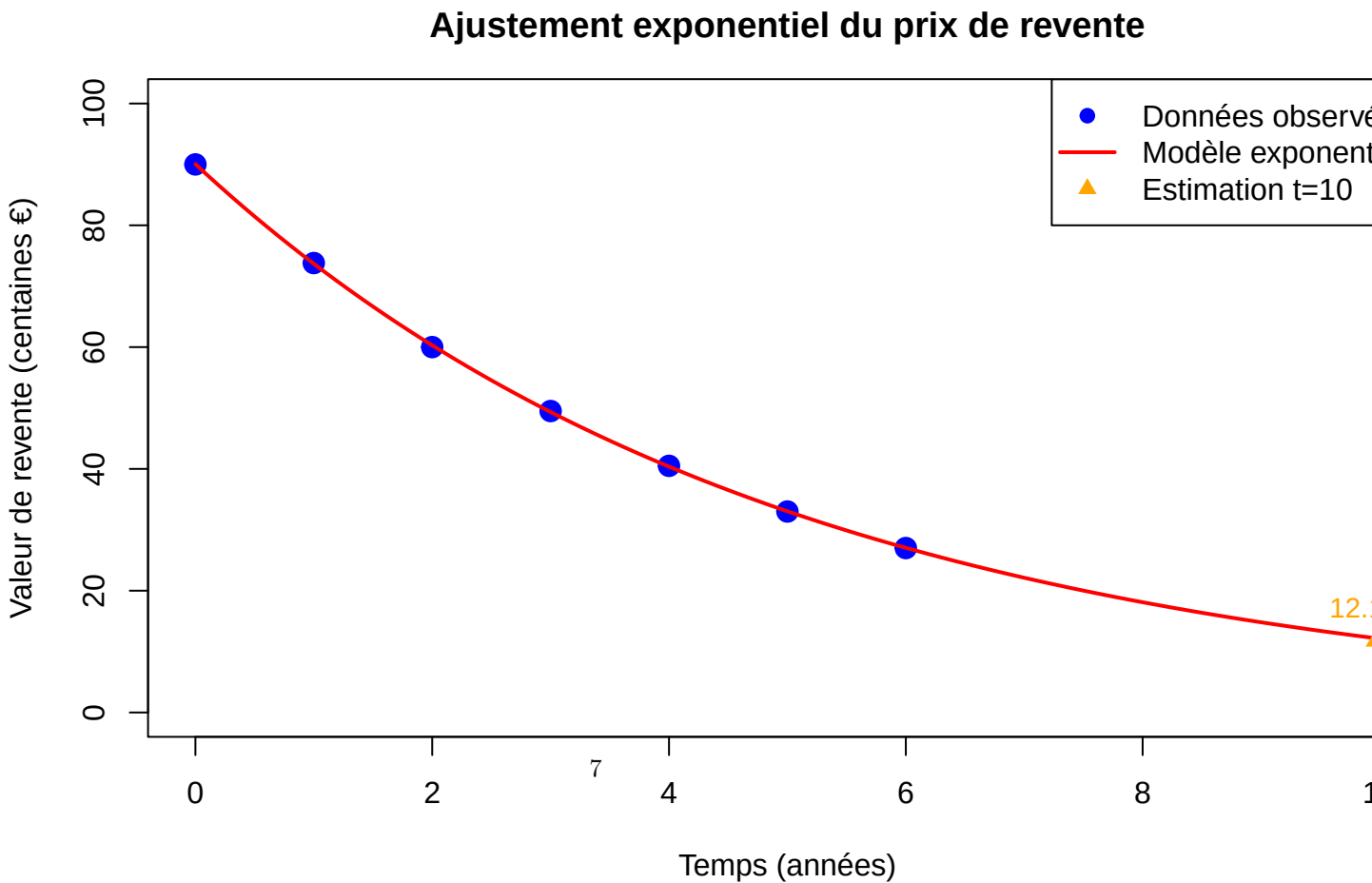
```
## Soit environ 1212 euros
```

1.4 Graphiques

```r
par(mfrow = c(2, 1), mar = c(4, 4, 3, 1))

# Graphique 1 : Ajustement linéaire sur Z = ln(X)
plot(t, Zt, pch = 19, col = "darkgreen", cex = 1.5,
     xlab = "Temps (années)", ylab = "Z = ln(X)",
     main = "Ajustement linéaire de Z = ln(Xt)")
t_seq <- seq(0, 10, 0.1)
lines(t_seq, a * t_seq + b, col = "red", lwd = 2)
abline(v = 10, lty = 3, col = "gray")
```

```r
points(10, a*10 + b, pch = 17, col = "blue", cex = 1.5)
legend("topright",
       legend = c("Données", "Droite MC", "Prédiction t=10"),
       col = c("darkgreen", "red", "blue"),
       pch = c(19, NA, 17), lwd = c(NA, 2, NA))

# Graphique 2 : Ajustement exponentiel
plot(t, Xt, pch = 19, col = "blue", cex = 1.5,
     xlab = "Temps (années)", ylab = "Valeur de revente (centaines €)",
     main = "Ajustement exponentiel du prix de revente",
     xlim = c(0, 10), ylim = c(0, 100))
lines(t_seq, A_coeff * exp(a * t_seq), col = "red", lwd = 2)
points(10, X10, pch = 17, col = "orange", cex = 1.5)
text(10, X10 + 5, paste0(round(X10, 1), " c.€"), col = "orange", cex = 0.9)
legend("topright",
       legend = c("Données observées", "Modèle exponentiel", "Estimation t=10"),
       col = c("blue", "red", "orange"),
       pch = c(19, NA, 17), lwd = c(NA, 2, NA))
```


Ajustement linéaire de Z = ln(Xt)
Données
Droite MC
Prédiction t=
Z = ln(X)
3.4
3.6
3.8
4.0
4.2
4.4
0
1
2
3
4
5
Temps (années)


Ajustement exponentiel du prix de revente
Données observé
Modèle exponent
Estimation t=10
Valeur de revente (centaines €)
0
20
40
60
80
100
0
2
4
6
8
Temps (années)

```r
par(mfrow = c(1, 1))
```

Interprétation : Le modèle exponentiel $X(t) = 90.04 \times e^{-0.2005t}$ ajuste très bien les données. La valeur de revente estimée à 10 ans est d'environ **12.1 centaines d'euros**, soit 1212 €.

2 Exercice 2 — Optimisation sans contrainte

2.1 Exercice 2.2.1 : $f(x, y) = e^{xy} - e^x + 2$

2.1.1 Recherche des points critiques

```r
# Définition de f
f1 <- function(x, y) exp(x * y) - exp(x) + 2

# Gradient de f
# df/dx = y*exp(xy) - exp(x)
# df/dy = x*exp(xy)
# df/dy = 0 => x = 0 (car exp(xy) != 0)
# df/dx = 0 avec x=0 => y*exp(0) - exp(0) = 0 => y - 1 = 0 => y = 1

cat("Résolution analytique du gradient = 0 :\n")
```

```
## Résolution analytique du gradient = 0 :
```

```r
cat("  df/dy = x*exp(xy) = 0  =>  x = 0\n")
```

```
##   df/dy = x*exp(xy) = 0  =>  x = 0
```

```r
cat("  df/dx = y*exp(xy) - exp(x) = 0  avec x=0  =>  y - 1 = 0  =>  y = 1\n\n")
```

```
##   df/dx = y*exp(xy) - exp(x) = 0  avec x=0  =>  y - 1 = 0  =>  y = 1
```

```r
x0 <- 0; y0 <- 1
cat("Point critique unique : (", x0, ",", y0, ")\n")
```

```
## Point critique unique : ( 0 , 1 )
```

```r
cat("Valeur de f(0,1) =", f1(x0, y0), "\n")
```

```
## Valeur de f(0,1) = 2
```

2.1.2 Matrice hessienne et nature du point

```r
# Hessienne analytique :
# d²f/dx² = y²*exp(xy) - exp(x)
# d²f/dxdy = exp(xy) + xy*exp(xy)
# d²f/dy² = x²*exp(xy)

H1 <- matrix(c(
  y0^2 * exp(x0*y0) - exp(x0),   exp(x0*y0) + x0*y0*exp(x0*y0),
  exp(x0*y0) + x0*y0*exp(x0*y0), x0^2 * exp(x0*y0)
), nrow = 2, byrow = TRUE)

cat("Hessienne en (0,1) :\n")
```

```
## Hessienne en (0,1) :
```

```r
print(H1)
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]    1    0
```

```r
vp1 <- eigen(H1)$values
cat("\nValeurs propres :", round(vp1, 6), "\n")
```

```
##
## Valeurs propres : 1 -1
```

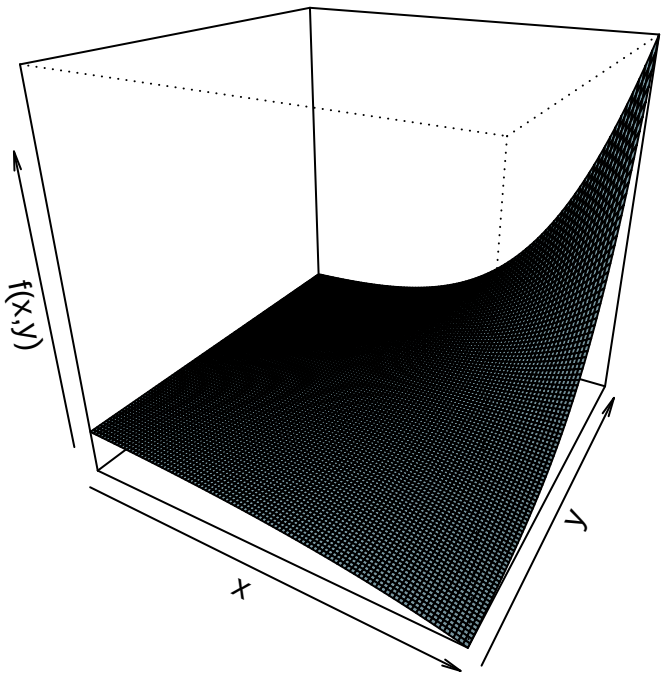
```r
if (all(vp1 > 0)) {
  cat("=> Hessienne définie positive : MINIMUM LOCAL\n")
} else if (all(vp1 < 0)) {
  cat("=> Hessienne définie négative : MAXIMUM LOCAL\n")
} else {
  cat("=> Hessienne indéfinie : POINT SELLE\n")
}
```

```
## => Hessienne indéfinie : POINT SELLE
```

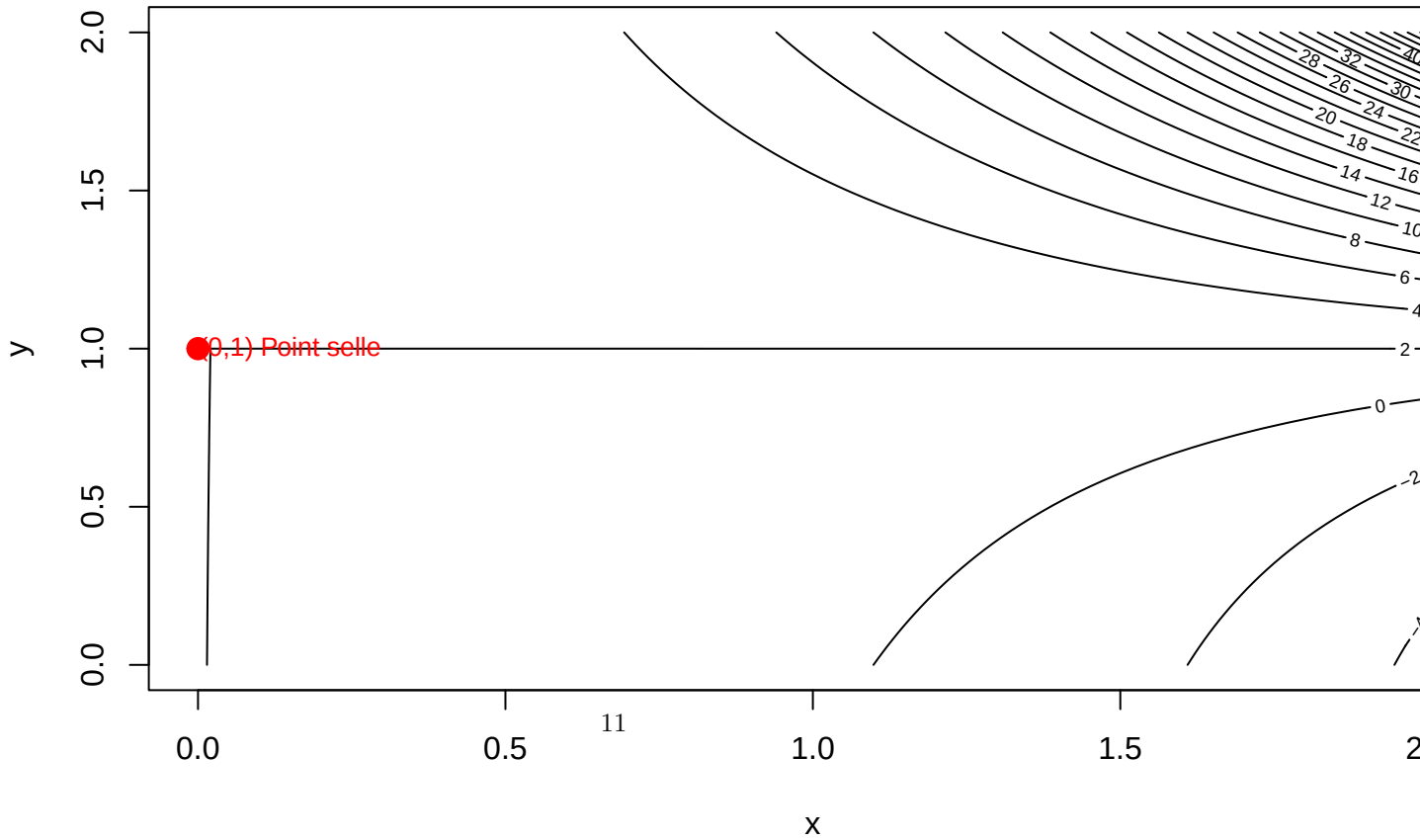
2.1.3 Tracé sur $[0, 2]^2$

```r
x_grid <- seq(0, 2, length.out = 100)
y_grid <- seq(0, 2, length.out = 100)
z_grid <- outer(x_grid, y_grid, f1)

par(mfrow = c(2, 1), mar = c(4, 4, 3, 1))

# Surface 3D
persp(x_grid, y_grid, z_grid,
      theta = 30, phi = 25, col = "lightblue", shade = 0.4,
      xlab = "x", ylab = "y", zlab = "f(x,y)",
      main = "Surface de f(x,y) = exp(xy) - exp(x) + 2")

# Courbes de niveau
contour(x_grid, y_grid, z_grid, nlevels = 25,
        xlab = "x", ylab = "y",
        main = "Courbes de niveau de f(x,y)")
points(x0, y0, pch = 19, col = "red", cex = 1.5)
text(x0 + 0.15, y0, "(0,1) Point selle", col = "red", cex = 0.8)
```

Surface de f(x,y) = exp(xy) − exp(x) + 2



Courbes de niveau de f(x,y)



```r
par(mfrow = c(1, 1))
```

Conclusion : La Hessienne en $(0, 1)$ est indéfinie (valeurs propres de signes opposés), donc $(0, 1)$ est un **point selle**.

2.2 Exercice 2.2.2 : $g(x, y, z) = (x + z^2)e^{x(y^2+z^2+1)}$

```r
g_func <- function(x, y, z) (x + z^2) * exp(x * (y^2 + z^2 + 1))

cat("Recherche des points critiques de g(x,y,z):\n\n")
```

```
## Recherche des points critiques de g(x,y,z):
```

```r
cat("dg/dy = 2xy(x+z²)exp(x(y²+z²+1)) = 0\n")
```

```
## dg/dy = 2xy(x+z²)exp(x(y²+z²+1)) = 0
```

```r
cat("dg/dz = [2z + 2xz(x+z²)]exp(x(y²+z²+1)) = 0\n")
```

```
## dg/dz = [2z + 2xz(x+z²)]exp(x(y²+z²+1)) = 0
```

```r
cat("dg/dx = [1 + (x+z²)(y²+z²+1)]exp(x(y²+z²+1)) = 0\n\n")
```

```
## dg/dx = [1 + (x+z²)(y²+z²+1)]exp(x(y²+z²+1)) = 0
```

```r
cat("Analyse:\n")
```

```
## Analyse:
```

```r
cat("- dg/dy = 0 avec exp(...) > 0 => 2xy(x+z²) = 0\n")
```

```
## - dg/dy = 0 avec exp(...) > 0 => 2xy(x+z²) = 0
```

```r
cat("- Si y = 0 et z = 0, alors dg/dx = [1 + x]exp(x) = 0 => x = -1\n\n")
```

```
## - Si y = 0 et z = 0, alors dg/dx = [1 + x]exp(x) = 0 => x = -1
```

```r
x0 <- -1; y0 <- 0; z0 <- 0
cat("Point critique : (", x0, ",", y0, ",", z0, ")\n")
```

```
## Point critique : ( -1 , 0 , 0 )
```

```r
cat("g(-1,0,0) =", g_func(x0, y0, z0), "\n\n")
```

```
## g(-1,0,0) = -0.3678794
```

```r
# Vérification : gradient en (-1,0,0)
E <- exp(-1)
# dg/dx en (-1,0,0) : [1 + (-1+0)*(0+0+1)]*exp(-1) = [1-1]*exp(-1) = 0
# dg/dy en (-1,0,0) : 2*(-1)*0*(-1+0)*exp(-1) = 0
# dg/dz en (-1,0,0) : [0 + 0]*exp(-1) = 0

# Hessienne en (-1,0,0), calculée analytiquement :
H2 <- exp(-1) * matrix(c(
  1, 0, 0,
  0, 2, 0,
  0, 0, 4
), nrow = 3, byrow = TRUE)

cat("Hessienne en (-1,0,0) :\n")
```

```
## Hessienne en (-1,0,0) :
```

```r
print(round(H2, 6))
```

```
##          [,1]     [,2]     [,3]
## [1,] 0.367879 0.000000 0.000000
## [2,] 0.000000 0.735759 0.000000
## [3,] 0.000000 0.000000 1.471518
```

```r
vp2 <- eigen(H2)$values
cat("\nValeurs propres :", round(vp2, 6), "\n")
```

```
##
## Valeurs propres : 1.471518 0.735759 0.367879
```

```r
if (all(vp2 > 0)) {
  cat("=> Hessienne définie positive : MINIMUM LOCAL\n")
  cat("=> g(-1,0,0) =", round(g_func(x0, y0, z0), 6), "est un minimum local\n")
} else if (all(vp2 < 0)) {
  cat("=> MAXIMUM LOCAL\n")
} else {
  cat("=> POINT SELLE\n")
}
```

```
## => Hessienne définie positive : MINIMUM LOCAL
## => g(-1,0,0) = -0.367879 est un minimum local
```

Conclusion : Toutes les valeurs propres de la Hessienne sont positives, donc $(-1, 0, 0)$ est un **minimum local** avec $g(-1, 0, 0) = -e^{-1} \approx -0.368$.

2.3 Exercice 2.2.3 : $h(x, y, z) = \ln(xyz) - \ln(x)\ln(y)\ln(z)$ sur $]0, +\infty[^3$

```r
h_func <- function(x, y, z) log(x*y*z) - log(x)*log(y)*log(z)

cat("Gradient de h :\n")
```

```
## Gradient de h :
```

```r
cat("dh/dx = (1 - ln(y)ln(z)) / x\n")
```

```
## dh/dx = (1 - ln(y)ln(z)) / x
```

```r
cat("dh/dy = (1 - ln(x)ln(z)) / y\n")
```

```
## dh/dy = (1 - ln(x)ln(z)) / y
```

```r
cat("dh/dz = (1 - ln(x)ln(y)) / z\n\n")
```

```
## dh/dz = (1 - ln(x)ln(y)) / z
```

```r
cat("Points critiques : dh/dx = dh/dy = dh/dz = 0\n")
```

```
## Points critiques : dh/dx = dh/dy = dh/dz = 0
```

```r
cat("=> 1 - ln(y)ln(z) = 0, 1 - ln(x)ln(z) = 0, 1 - ln(x)ln(y) = 0\n")
```

```
## => 1 - ln(y)ln(z) = 0, 1 - ln(x)ln(z) = 0, 1 - ln(x)ln(y) = 0
```

```r
cat("Par symétrie : ln(x) = ln(y) = ln(z) = c\n")
```

```
## Par symétrie : ln(x) = ln(y) = ln(z) = c
```

```r
cat("=> c² = 1  =>  c = 1 ou c = -1\n")
```

```
## => c² = 1  =>  c = 1 ou c = -1
```

```r
cat("=> (e, e, e) ou (1/e, 1/e, 1/e)\n\n")
```

```
## => (e, e, e) ou (1/e, 1/e, 1/e)
```

```r
# Point critique 1 : (e, e, e)
cat("===== Point critique 1 : (e, e, e) =====\n")
```

```
## ===== Point critique 1 : (e, e, e) =====
```

```r
x1 <- exp(1); y1 <- exp(1); z1 <- exp(1)
cat("h(e,e,e) =", h_func(x1, y1, z1), "\n\n")
```

```
## h(e,e,e) = 2
```

```r
# Hessienne en (e,e,e)
# d²h/dx² = -(1 - ln(y)ln(z))/x² = 0/e² = 0
# d²h/dxdy = -ln(z)/(xy) = -1/e²
H_eee <- matrix(c(
  0,          -1/exp(2), -1/exp(2),
  -1/exp(2),  0,         -1/exp(2),
  -1/exp(2), -1/exp(2),  0
), nrow = 3, byrow = TRUE)

cat("Hessienne en (e,e,e) :\n")
```

```
## Hessienne en (e,e,e) :
```

```r
print(round(H_eee, 6))
```

```
##           [,1]      [,2]      [,3]
## [1,]  0.000000 -0.135335 -0.135335
## [2,] -0.135335  0.000000 -0.135335
## [3,] -0.135335 -0.135335  0.000000
```

```r
vp_eee <- eigen(H_eee)$values
cat("\nValeurs propres :", round(vp_eee, 6), "\n")
```

```
##
## Valeurs propres : 0.135335 0.135335 -0.270671
```

```r
if (any(vp_eee > 1e-10) && any(vp_eee < -1e-10)) {
  cat("=> Hessienne indéfinie : POINT SELLE\n")
}
```

```
## => Hessienne indéfinie : POINT SELLE
```

```r
# Point critique 2 : (1/e, 1/e, 1/e)
cat("\n===== Point critique 2 : (1/e, 1/e, 1/e) =====\n")
```

```
##
## ===== Point critique 2 : (1/e, 1/e, 1/e) =====
```

```r
x2 <- exp(-1); y2 <- exp(-1); z2 <- exp(-1)
cat("h(1/e,1/e,1/e) =", h_func(x2, y2, z2), "\n\n")
```

```
## h(1/e,1/e,1/e) = -2
```

```r
H_inv <- matrix(c(
  0,        exp(2), exp(2),
  exp(2),   0,      exp(2),
  exp(2),   exp(2), 0
), nrow = 3, byrow = TRUE)

cat("Hessienne en (1/e,1/e,1/e) :\n")
```

```
## Hessienne en (1/e,1/e,1/e) :
```

```r
print(round(H_inv, 6))
```

```
##          [,1]     [,2]     [,3]
## [1,] 0.000000 7.389056 7.389056
## [2,] 7.389056 0.000000 7.389056
## [3,] 7.389056 7.389056 0.000000
```

```r
vp_inv <- eigen(H_inv)$values
cat("\nValeurs propres :", round(vp_inv, 6), "\n")
```

```
##
## Valeurs propres : 14.77811 -7.389056 -7.389056
```

```r
if (any(vp_inv > 1e-10) && any(vp_inv < -1e-10)) {
  cat("=> Hessienne indéfinie : POINT SELLE\n")
}
```

```
## => Hessienne indéfinie : POINT SELLE
```

Conclusion : Les deux points critiques (e, e, e) et $(1/e, 1/e, 1/e)$ sont des **points selles**. La fonction h n'admet pas d'extremum local sur $]0, +\infty[^3$.

3 Exercice 3 — Optimisation sous contraintes

3.1 Exercice 3.2.1 : $f(x, y) = xy$ sous $g(x, y) = x - y + 6 = 0$

```r
cat("Lagrangien : L(x,y, ) = xy -  (x - y + 6)\n\n")
```

```
## Lagrangien : L(x,y, ) = xy -  (x - y + 6)
```

```r
cat("Conditions nécessaires (gradient = 0) :\n")
```

```
## Conditions nécessaires (gradient = 0) :
```

```r
cat("  dL/dx = y -   = 0       =>   = y\n")
```

```
##   dL/dx = y -   = 0       =>   = y
```

```r
cat("  dL/dy = x +   = 0       => x = -  = -y\n")
```

```
##   dL/dy = x +   = 0       => x = -  = -y
```

```r
cat("  dL/d  = -(x - y + 6) = 0 => x - y + 6 = 0\n\n")
```

```
##   dL/d  = -(x - y + 6) = 0 => x - y + 6 = 0
```

```r
cat("En substituant x = -y dans la contrainte : -y - y + 6 = 0 => y = 3\n")
```

```
## En substituant x = -y dans la contrainte : -y - y + 6 = 0 => y = 3
```

```r
cat("Donc x = -3,   = 3\n\n")
```

```
## Donc x = -3,   = 3
```

```r
x0 <- -3; y0 <- 3; lambda0 <- 3
cat("Point critique : (", x0, ",", y0, "),   =", lambda0, "\n")
```

```
## Point critique : ( -3 , 3 ),   = 3
```

```r
cat("f(-3, 3) =", x0 * y0, "\n\n")
```

```
## f(-3, 3) = -9
```

```r
# Hessienne bordée
cat("Hessienne du Lagrangien :\n")
```

```
## Hessienne du Lagrangien :
```

```r
HL <- matrix(c(0, 1, 1, 0), nrow = 2)
print(HL)
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]    1    0
```

```r
cat("\nJacobienne de la contrainte : Dg = (1, -1)\n")
```

```
##
## Jacobienne de la contrainte : Dg = (1, -1)
```

```r
H_bordee <- rbind(
  c(0, 1, -1),
  c(1, 0,  1),
  c(-1, 1, 0)
)
cat("\nHessienne bordée H :\n")
```

```
##
## Hessienne bordée H :
```

```r
print(H_bordee)
```

```
##      [,1] [,2] [,3]
## [1,]    0    1   -1
## [2,]    1    0    1
## [3,]   -1    1    0
```

```r
det_H <- det(H_bordee)
cat("\ndet(H) =", det_H, "\n")
```

```
##
## det(H) = -2
```

```r
# n=2, p=1 => n-p = 1 dernier mineur principal
# det(H) et signe de (-1)^p = (-1)^1 = -1
if (det_H < 0) {
  cat("det(H) < 0, du même signe que (-1)^p = -1\n")
  cat("=> MINIMUM LOCAL sous contrainte\n")
} else if (det_H > 0) {
  cat("det(H) > 0, du même signe que (-1)^n = (-1)^2 = 1\n")
  cat("=> MAXIMUM LOCAL sous contrainte\n")
}
```

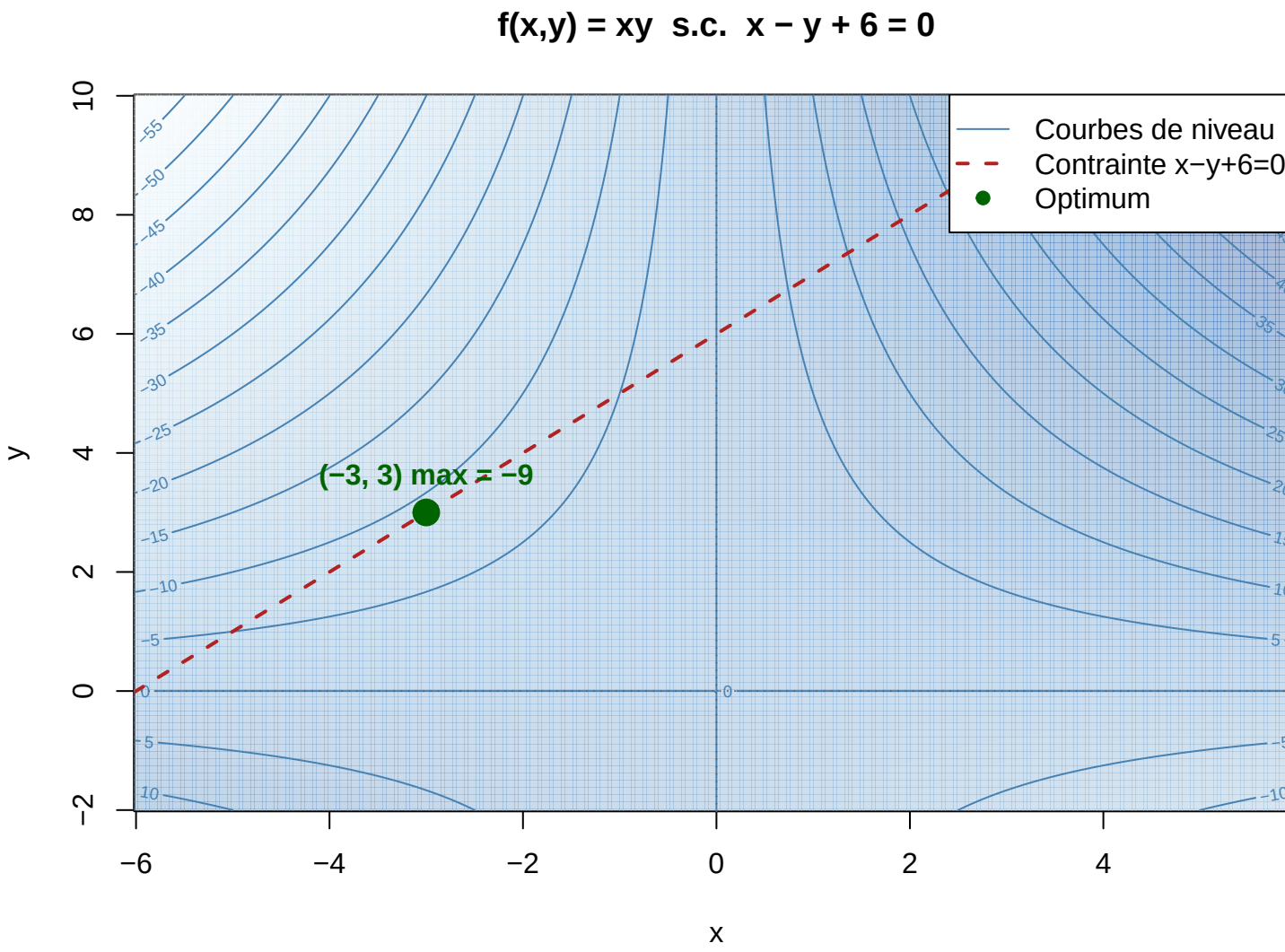
```
## det(H) < 0, du même signe que (-1)^p = -1
## => MINIMUM LOCAL sous contrainte
```

```r
cat("f(-3,3) =", x0*y0, "est un maximum local sous contrainte.\n")
```

```
## f(-3,3) = -9 est un maximum local sous contrainte.
```

3.1.1 Tracé

```r
f3_1 <- function(x, y) x * y
x_seq <- seq(-6, 6, length.out = 300)
y_seq <- seq(-2, 10, length.out = 300)
z_mat <- outer(x_seq, y_seq, f3_1)

image(x_seq, y_seq, z_mat,
      col = hcl.colors(100, "Blues", rev = TRUE, alpha = 0.4),
      xlab = "x", ylab = "y",
      main = "f(x,y) = xy  s.c.  x - y + 6 = 0")
contour(x_seq, y_seq, z_mat, nlevels = 20, col = "steelblue", add = TRUE, drawlabels = TRUE)
# Contrainte y = x + 6
abline(a = 6, b = 1, col = "firebrick", lwd = 2, lty = 2)
points(-3, 3, pch = 19, col = "darkgreen", cex = 2)
text(-3, 3.6, "(-3, 3) max = -9", col = "darkgreen", font = 2)
abline(h = 0, v = 0, col = "gray40", lty = 3)
legend("topright",
       legend = c("Courbes de niveau", "Contrainte x-y+6=0", "Optimum"),
       col = c("steelblue", "firebrick", "darkgreen"),
       lty = c(1, 2, NA), pch = c(NA, NA, 19), lwd = c(1, 2, NA), bg = "white")
```



3.2 Exercice 3.2.2 : $f(x, y, z) = x^2 + y^2 + z^2$ sous $g_1 = x + y + z - 1 = 0$ et $g_2 = z = 0$

```
cat("Lagrangien : L = x²+y²+z² -  (x+y+z-1) -  (z)\n\n")
```

```
## Lagrangien : L = x²+y²+z² -  (x+y+z-1) -  (z)
```

```
cat("dL/dx = 2x -   = 0       => x =  /2\n")
```

```
## dL/dx = 2x -   = 0       => x =  /2
```

```
cat("dL/dy = 2y -   = 0       => y =  /2\n")
```

```
## dL/dy = 2y -   = 0       => y =  /2
```

```r
cat("dL/dz = 2z -   -   = 0  => z = ( + )/2\n")
```

```
## dL/dz = 2z -   -   = 0  => z = ( + )/2
```

```r
cat("g  : z = 0  =>    = - \n")
```

```
## g  : z = 0  =>    = -
```

```r
cat("g  : x + y + z = 1  =>   /2 +  /2 + 0 = 1  =>    = 1\n\n")
```

```
## g  : x + y + z = 1  =>   /2 +  /2 + 0 = 1  =>    = 1
```

```r
x0 <- 1/2; y0 <- 1/2; z0 <- 0
cat("Point critique : (", x0, ",", y0, ",", z0, ")\n")
```

```
## Point critique : ( 0.5 , 0.5 , 0 )
```

```r
cat("f(1/2, 1/2, 0) =", x0^2 + y0^2 + z0^2, "\n\n")
```

```
## f(1/2, 1/2, 0) = 0.5
```

```r
# Hessienne bordée
HL <- diag(2, 3) # Hessienne du Lagrangien = 2I
Dg <- rbind(c(1,1,1), c(0,0,1))
H_bordee <- rbind(
  cbind(matrix(0, 2, 2), Dg),
  cbind(t(Dg), HL)
)
cat("Hessienne bordée (5×5) :\n")
```

```
## Hessienne bordée (5×5) :
```

```r
print(H_bordee)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    0    0    1    1    1
## [2,]    0    0    0    0    1
## [3,]    1    0    2    0    0
## [4,]    1    0    0    2    0
## [5,]    1    1    0    0    2
```

```r
det_H <- det(H_bordee)
cat("\ndet(H) =", det_H, "\n")
```

```
##
## det(H) = 4
```

```r
# n=3, p=2 => n-p=1 dernier mineur principal
# det(H) du signe de (-1)^p = (-1)^2 = 1 => minimum
cat("(-1)^p = (-1)^2 = 1. det(H) =", det_H, "> 0 => MINIMUM LOCAL\n")
```

```
## (-1)^p = (-1)^2 = 1. det(H) = 4 > 0 => MINIMUM LOCAL
```

```r
cat("=> f(1/2, 1/2, 0) = 1/2 est un minimum sous contraintes.\n")
```

```
## => f(1/2, 1/2, 0) = 1/2 est un minimum sous contraintes.
```

3.3 Exercice 3.2.3 : $f = x^2+y^2+z^2$ sous $g_1 = x^2+y^2+2z-6 = 0$ et $g_2 = x-y-z = 0$

```r
cat("Lagrangien : L = x²+y²+z² -  (x²+y²+2z-6) -  (x-y-z)\n\n")
```

```
## Lagrangien : L = x²+y²+z² -  (x²+y²+2z-6) -  (x-y-z)
```

```r
cat("dL/dx = 2x - 2 x -   = 0\n")
```

```
## dL/dx = 2x - 2 x -   = 0
```

```r
cat("dL/dy = 2y - 2 y +   = 0\n")
```

```
## dL/dy = 2y - 2 y +   = 0
```

```r
cat("dL/dz = 2z - 2  +   = 0\n\n")
```

```
## dL/dz = 2z - 2  +   = 0
```

```r
cat("Résolution numérique des points critiques :\n")
```

```
## Résolution numérique des points critiques :
```

```r
# On cherche les solutions du système
# En additionnant eq1 et eq2 : 2(x+y)(1- ) = 0
# Cas 1:   = 1 => eq3: 2z - 2 +   = 0 =>   = 2-2z
# eq1: 2x - 2x - (2-2z) = 0 => 2z-2 = 0 => z = 1
# g2: x - y - 1 = 0 => x = y+1
# g1: x²+y²+2-6 = 0 => (y+1)²+y² = 4 => 2y²+2y-3 = 0
# y = (-2 ± sqrt(4+24))/4 = (-2 ± sqrt(28))/4
y_sol <- (-2 + sqrt(28))/4
x_sol <- y_sol + 1
z_sol <- 1
cat("\nCas   = 1 :\n")
```

```
##
## Cas   = 1 :
```

```r
cat("  Point 1 : (", round(x_sol,4), ",", round(y_sol,4), ",", z_sol, ")\n")
```

```
##   Point 1 : ( 1.8229 , 0.8229 , 1 )
```

```r
cat("  f =", round(x_sol^2 + y_sol^2 + z_sol^2, 4), "\n")
```

```
##   f = 5
```

```r
y_sol2 <- (-2 - sqrt(28))/4
x_sol2 <- y_sol2 + 1
cat("  Point 2 : (", round(x_sol2,4), ",", round(y_sol2,4), ",", z_sol, ")\n")
```

```
##   Point 2 : ( -0.8229 , -1.8229 , 1 )
```

```r
cat("  f =", round(x_sol2^2 + y_sol2^2 + z_sol^2, 4), "\n")
```

```
##   f = 5
```

```r
# Cas 2 : x + y = 0 => y = -x
# g2: x - (-x) - z = 0 => z = 2x
# g1: x² + x² + 4x - 6 = 0 => 2x² + 4x - 6 = 0 => x² + 2x - 3 = 0
# x = (-2 ± sqrt(4+12))/2 = (-2 ± 4)/2
cat("\nCas x + y = 0 :\n")
```

```
##
## Cas x + y = 0 :
```

```r
x_a <- 1; y_a <- -1; z_a <- 2
cat("  Point 3 : (", x_a, ",", y_a, ",", z_a, ")\n")
```

```
##   Point 3 : ( 1 , -1 , 2 )
```

```r
cat("  f =", x_a^2 + y_a^2 + z_a^2, "\n")
```

```
##   f = 6
```

```r
x_b <- -3; y_b <- 3; z_b <- -6
cat("  Point 4 : (", x_b, ",", y_b, ",", z_b, ")\n")
```

```
##   Point 4 : ( -3 , 3 , -6 )
```

```r
cat("  f =", x_b^2 + y_b^2 + z_b^2, "\n")
```

```
##   f = 54
```

```r
# Nature des points
cat("\nAnalyse de la nature des points par Hessienne bordée :\n")
```

```
##
## Analyse de la nature des points par Hessienne bordée :
```

```r
for (pt in list(c(x_a, y_a, z_a), c(x_b, y_b, z_b))) {
  Dg_pt <- rbind(
    c(2*pt[1], 2*pt[2], 2),
    c(1, -1, -1)
  )
  HL_pt <- diag(2, 3) # Hessienne constante = 2I
  H_bord <- rbind(
    cbind(matrix(0, 2, 2), Dg_pt),
    cbind(t(Dg_pt), HL_pt)
  )
  d <- det(H_bord)
  cat("  Point (", pt[1], ",", pt[2], ",", pt[3], ") : det(H) =", round(d, 4), "\n")
}
```

```
##   Point ( 1 , -1 , 2 ) : det(H) = 64
##   Point ( -3 , 3 , -6 ) : det(H) = 64
```

```r
cat("=> (1,-1,2) avec f = 6 : minimum local\n")
```

```
## => (1,-1,2) avec f = 6 : minimum local
```

```r
cat("=> (-3,3,-6) avec f = 54 : maximum local\n")
```

```
## => (-3,3,-6) avec f = 54 : maximum local
```

3.4 Exercice 3.2.4 : $f(x, y) = x^2y$ sous $2x^2 + y^2 = 3$

```r
cat("Lagrangien : L = x²y -  (2x² + y² - 3)\n\n")
```

```
## Lagrangien : L = x²y -  (2x² + y² - 3)
```

```r
cat("dL/dx = 2xy - 4 x = 0  => 2x(y - 2 ) = 0\n")
```

```
## dL/dx = 2xy - 4 x = 0  => 2x(y - 2 ) = 0
```

```r
cat("dL/dy = x² - 2 y = 0   =>   = x²/(2y)\n")
```

```
## dL/dy = x² - 2 y = 0   =>   = x²/(2y)
```

```r
cat("Contrainte : 2x² + y² = 3\n\n")
```

```
## Contrainte : 2x² + y² = 3
```

```r
# Cas x = 0 => y² = 3 => y = ±√3, f = 0
# Cas y = 2  =>   = x²/(2y) => y = x²/y => y² = x²
# => y = ±x. Avec 2x² + x² = 3 => x² = 1

pts <- data.frame(
  x = c(0, 0, 1, -1, 1, -1),
  y = c(sqrt(3), -sqrt(3), 1, 1, -1, -1),
  stringsAsFactors = FALSE
)
pts$f <- pts$x^2 * pts$y

cat("Points critiques :\n")
```

```
## Points critiques :
```

```r
for (i in 1:nrow(pts)) {
  cat(sprintf("  (%6.3f, %6.3f) : f = %6.3f\n", pts$x[i], pts$y[i], pts$f[i]))
}
```

```
##   ( 0.000,  1.732) : f =  0.000
##   ( 0.000, -1.732) : f = -0.000
##   ( 1.000,  1.000) : f =  1.000
##   (-1.000,  1.000) : f =  1.000
##   ( 1.000, -1.000) : f = -1.000
##   (-1.000, -1.000) : f = -1.000
```

```r
cat("\n=> Maximum : f = 1 en (±1, 1)\n")
```

```
##
## => Maximum : f = 1 en (±1, 1)
```

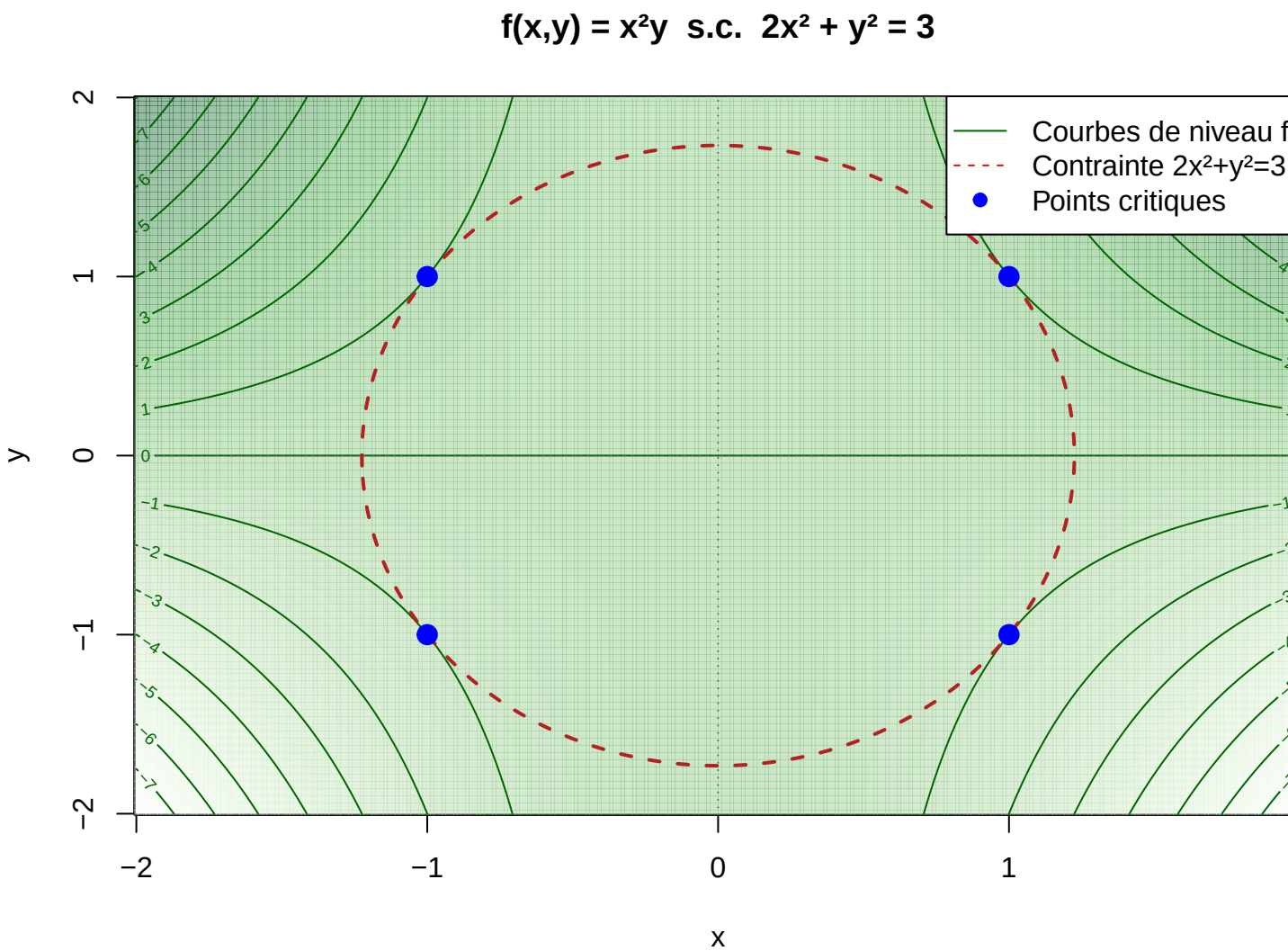
```r
cat("=> Minimum : f = -1 en (±1, -1)\n")
```

```
## => Minimum : f = -1 en (±1, -1)
```

3.4.1 Tracé

```r
f3_4 <- function(x, y) x^2 * y
g3_4 <- function(x, y) 2*x^2 + y^2 - 3

x_seq <- seq(-2, 2, length.out = 300)
y_seq <- seq(-2, 2, length.out = 300)
z_mat <- outer(x_seq, y_seq, f3_4)
```

```r
image(x_seq, y_seq, z_mat,
      col = hcl.colors(100, "Greens", rev = TRUE, alpha = 0.4),
      xlab = "x", ylab = "y",
      main = "f(x,y) = x²y  s.c.  2x² + y² = 3")
contour(x_seq, y_seq, z_mat, nlevels = 20, col = "darkgreen", add = TRUE, drawlabels = TRUE)
g_mat <- outer(x_seq, y_seq, g3_4)
contour(x_seq, y_seq, g_mat, levels = 0, col = "firebrick", lwd = 2, lty = 2,
        add = TRUE, drawlabels = FALSE)
points(c(1,-1,1,-1), c(1,1,-1,-1), pch = 19, col = "blue", cex = 1.5)
abline(h = 0, v = 0, col = "gray40", lty = 3)
legend("topright",
       legend = c("Courbes de niveau f", "Contrainte 2x²+y²=3", "Points critiques"),
       col = c("darkgreen", "firebrick", "blue"),
       lty = c(1, 2, NA), pch = c(NA, NA, 19), bg = "white")
```



3.5 Exercice 3.2.5 : $f(x,y) = \frac{x^2y^2}{x^2+y^2} + 2x^2 + 2y^2 + 4xy + 5x + 5y + 20$ sous $2x^2 + 2y^2 + 4xy - 8 = 3$

```
f3_5 <- function(x, y) {
  r <- x^2 + y^2
  ifelse(r == 0, NA, x^2*y^2/r + 2*x^2 + 2*y^2 + 4*x*y + 5*x + 5*y + 20)
}
g3_5 <- function(x, y) 2*x^2 + 2*y^2 + 4*x*y - 11 # = 0

cat("Contrainte : 2x² + 2y² + 4xy = 11  <=>  2(x+y)² = 11\n")
```

```
## Contrainte : 2x² + 2y² + 4xy = 11  <=>  2(x+y)² = 11
```

```
cat("=> (x+y)² = 11/2  =>  x+y = ±√(11/2)\n\n")
```

```
## => (x+y)² = 11/2  =>  x+y = ±√(11/2)
```

```
cat("Sur la contrainte : 2(x+y)² = 11, donc 4xy = 11 - 2x² - 2y²\n")
```

```
## Sur la contrainte : 2(x+y)² = 11, donc 4xy = 11 - 2x² - 2y²
```

```
cat("Analyse numérique des points critiques...\n\n")
```

```
## Analyse numérique des points critiques...
```

```
# Points possibles : x + y = sqrt(11/2) ou x + y = -sqrt(11/2)
s <- sqrt(11/2)
cat("Cas x+y =", round(s,4), ":\n")
```

```
## Cas x+y = 2.3452 :
```

```
cat("  Exemples : x = 0, y =", round(s,4), " => f =", round(f3_5(0, s),4), "\n")
```

```
##   Exemples : x = 0, y = 2.3452  => f = 42.726
```

```
cat("  x = y =", round(s/2,4), " => f =", round(f3_5(s/2, s/2),4), "\n\n")
```

```
##   x = y = 1.1726  => f = 43.4135
```

```
cat("Cas x+y =", round(-s,4), ":\n")
```

```
## Cas x+y = -2.3452 :
```

```
cat("  x = 0, y =", round(-s,4), " => f =", round(f3_5(0, -s),4), "\n")
```

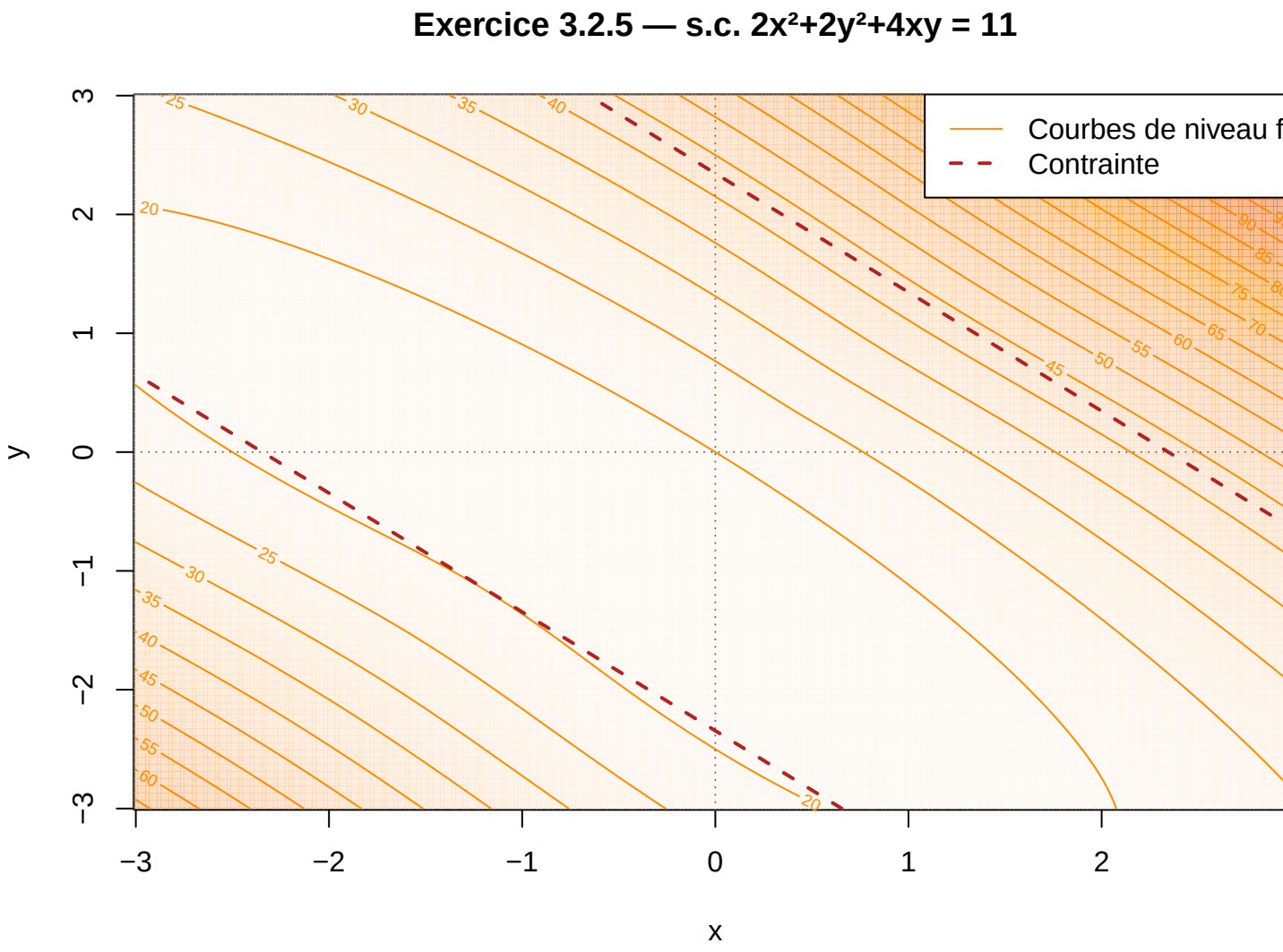
```
##   x = 0, y = -2.3452  => f = 19.274
```

```r
cat("  x = y =", round(-s/2,4), " => f =", round(f3_5(-s/2, -s/2),4), "\n")
```

```
##   x = y = -1.1726  => f = 19.9615
```

3.5.1 Tracé

```r
x_seq <- seq(-3, 3, length.out = 300)
y_seq <- seq(-3, 3, length.out = 300)
z_mat <- outer(x_seq, y_seq, f3_5)
z_mat[abs(z_mat) > 200] <- NA

image(x_seq, y_seq, z_mat,
      col = hcl.colors(100, "Oranges", rev = TRUE, alpha = 0.4),
      xlab = "x", ylab = "y",
      main = "Exercice 3.2.5 - s.c. 2x²+2y²+4xy = 11")
contour(x_seq, y_seq, z_mat, nlevels = 20, col = "darkorange", add = TRUE, drawlabels = TRUE)
g_mat <- outer(x_seq, y_seq, g3_5)
contour(x_seq, y_seq, g_mat, levels = 0, col = "firebrick", lwd = 2, lty = 2,
        add = TRUE, drawlabels = FALSE)
abline(h = 0, v = 0, col = "gray40", lty = 3)
legend("topright",
       legend = c("Courbes de niveau f", "Contrainte"),
       col = c("darkorange", "firebrick"), lty = c(1, 2), lwd = c(1, 2), bg = "white")
```


Exercice 3.2.5 — s.c. 2x²+2y²+4xy = 11
Courbes de niveau f
Contrainte
x
y
−3
−2
−1
0
1
2
3
20
25
30
35
40
45
50
55
60
65
70
75
80
85
90

4 Exercice 4 — Décomposition en valeurs singulières et pseudo-inverse

4.1 Exercice 4.2 : SVD — Cas 1

4.1.1 SVD de la matrice A

```r
cat("===== DÉCOMPOSITION SVD DE A (4×4) =====\n\n")
```

```
## ===== DÉCOMPOSITION SVD DE A (4×4) =====
```

```r
A <- matrix(c(-18, 13, -4, 4,
              2, 19, -4, 12,
              -14, 11, -12, 8,
              -2, 21, 4, 8), nrow = 4, byrow = TRUE)
cat("Matrice A :\n"); print(A)
```

```
## Matrice A :

##      [,1] [,2] [,3] [,4]
## [1,]  -18   13   -4    4
## [2,]    2   19   -4   12
## [3,]  -14   11  -12    8
## [4,]   -2   21    4    8
```

```r
# Étape 1 : A^T A
AtA <- t(A) %*% A
cat("\nA^T A :\n"); print(AtA)
```

```
##
## A^T A :

##      [,1] [,2] [,3] [,4]
## [1,]  528 -392  224 -176
## [2,] -392 1092 -176  536
## [3,]  224 -176  192 -128
## [4,] -176  536 -128  288
```

```r
# Étape 2 : Diagonalisation de A^T A
eig_A <- eigen(AtA)
cat("\nValeurs propres de A^T A :\n"); print(round(eig_A$values, 4))
```

```
##
## Valeurs propres de A^T A :

## [1] 1600  400  100    0
```

```r
# Étape 3 : Valeurs singulières
sv_A <- sqrt(abs(eig_A$values))
ordre <- order(sv_A, decreasing = TRUE)
sv_A <- sv_A[ordre]
V_A <- eig_A$vectors[, ordre]
cat("\nValeurs singulières de A (triées) :\n"); print(round(sv_A, 4))
```

```
##
## Valeurs singulières de A (triées) :

## [1] 40 20 10  0
```

```r
tolerance <- 1e-10
rang_A <- sum(sv_A > tolerance)
cat("Rang de A :", rang_A, "\n")
```

```
## Rang de A : 4
```

```r
# Étape 4 : Matrice V
cat("\nMatrice V :\n"); print(round(V_A, 6))
```

```
##
## Matrice V :

##      [,1] [,2] [,3] [,4]
## [1,]  0.4  0.8  0.4 -0.2
## [2,] -0.8  0.4 -0.2 -0.4
## [3,]  0.2  0.4 -0.8  0.4
## [4,] -0.4  0.2  0.4  0.8
```

```r
# Étape 5 : Matrice U
Ur_A <- matrix(0, nrow = nrow(A), ncol = rang_A)
for (i in 1:rang_A) {
  Av <- A %*% V_A[, i]
  Ur_A[, i] <- Av / sv_A[i]
}
if (rang_A < nrow(A)) {
  U_A <- qr.Q(qr(Ur_A), complete = TRUE)
} else {
  U_A <- Ur_A
}
cat("\nMatrice U :\n"); print(round(U_A, 6))
```

```
##
## Matrice U :

##      [,1] [,2] [,3] [,4]
## [1,] -0.5 -0.5 -0.5    0
## [2,] -0.5  0.5  0.5    0
## [3,] -0.5 -0.5  0.5    0
## [4,] -0.5  0.5 -0.5    0
```

```r
# Étape 6 : Sigma
Sigma_A <- matrix(0, nrow(A), ncol(A))
for (i in 1:min(rang_A, nrow(A), ncol(A))) Sigma_A[i, i] <- sv_A[i]
cat("\nΣ :\n"); print(round(Sigma_A, 4))
```

```
##
## Σ :

##      [,1] [,2] [,3] [,4]
## [1,]   40    0    0    0
## [2,]    0   20    0    0
## [3,]    0    0   10    0
## [4,]    0    0    0    0
```

```r
# Vérification
A_recon <- U_A %*% Sigma_A %*% t(V_A)
cat("\nA reconstruite (U Σ V^T) :\n"); print(round(A_recon, 4))
```

```
##
## A reconstruite (U Σ V^T) :

##      [,1] [,2] [,3] [,4]
## [1,]  -18   13   -4    4
## [2,]    2   19   -4   12
## [3,]  -14   11  -12    8
## [4,]   -2   21    4    8
```

```r
cat("Erreur de reconstruction :", round(norm(A - A_recon, "F"), 10), "\n")
```

```
## Erreur de reconstruction : 0
```

4.1.2 SVD de la matrice B

```r
cat("===== DÉCOMPOSITION SVD DE B (4×5) =====\n\n")
```

```
## ===== DÉCOMPOSITION SVD DE B (4×5) =====
```

```r
B <- matrix(c(6, -8, -4, 5, -4,
              2, 7, -5, -6, 4,
              0, -1, -8, 2, 2,
              -1, -2, 4, 4, -8), nrow = 4, byrow = TRUE)
cat("Matrice B :\n"); print(B)
```

```
## Matrice B :

##      [,1] [,2] [,3] [,4] [,5]
## [1,]    6   -8   -4    5   -4
## [2,]    2    7   -5   -6    4
## [3,]    0   -1   -8    2    2
## [4,]   -1   -2    4    4   -8
```

```r
BtB <- t(B) %*% B
eig_B <- eigen(BtB)
sv_B <- sqrt(abs(eig_B$values))
ordre_B <- order(sv_B, decreasing = TRUE)
sv_B <- sv_B[ordre_B]
V_B <- eig_B$vectors[, ordre_B]
cat("\nValeurs singulières de B :\n"); print(round(sv_B, 4))
```

```
##
## Valeurs singulières de B :

## [1] 16.4580 12.1595  4.8710  4.3072  0.0000
```

```r
rang_B <- sum(sv_B > tolerance)
rang_B <- min(rang_B, nrow(B), ncol(B))  # Le rang ne peut pas dépasser min(m,n)
cat("Rang de B :", rang_B, "\n")
```

```
## Rang de B : 4
```

```r
Ur_B <- matrix(0, nrow(B), rang_B)
for (i in 1:rang_B) {
  Bv <- B %*% V_B[, i]
  Ur_B[, i] <- Bv / sv_B[i]
}
if (rang_B < nrow(B)) {
  U_B <- qr.Q(qr(Ur_B), complete = TRUE)
} else {
  U_B <- Ur_B
}

Sigma_B <- matrix(0, nrow(B), ncol(B))
for (i in 1:min(rang_B, nrow(B), ncol(B))) Sigma_B[i, i] <- sv_B[i]

B_recon <- U_B %*% Sigma_B %*% t(V_B)
cat("\nB reconstruite :\n"); print(round(B_recon, 4))
```

```
##
## B reconstruite :

##      [,1] [,2] [,3] [,4] [,5]
## [1,]    6   -8   -4    5   -4
## [2,]    2    7   -5   -6    4
## [3,]    0   -1   -8    2    2
## [4,]   -1   -2    4    4   -8
```

```r
cat("Erreur :", round(norm(B - B_recon, "F"), 10), "\n")
```

```
## Erreur : 0
```

4.2 Exercice 4.3 : SVD — Cas 2 (Pseudo-inverse et résolution)

4.2.1 Pseudo-inverse de A

```r
cat("===== PSEUDO-INVERSE DE A (4×5) =====\n\n")
```

```
## ===== PSEUDO-INVERSE DE A (4×5) =====
```

```r
A2 <- matrix(c(-3, -3, -6, 6, 1,
               -1, -1, -1, 1, -2,
                0,  0, -1, 1, -1,
                0,  0,  1, 1, -1), nrow = 4, byrow = TRUE)
cat("Matrice A :\n"); print(A2)
```

```
## Matrice A :

##      [,1] [,2] [,3] [,4] [,5]
## [1,]   -3   -3   -6    6    1
## [2,]   -1   -1   -1    1   -2
## [3,]    0    0   -1    1   -1
## [4,]    0    0    1    1   -1
```

```r
AtA2 <- t(A2) %*% A2
eig2 <- eigen(AtA2)
sv2 <- sqrt(abs(eig2$values))
ord2 <- order(sv2, decreasing = TRUE)
sv2 <- sv2[ord2]
V2 <- eig2$vectors[, ord2]

rang2 <- min(sum(sv2 > 1e-10), nrow(A2), ncol(A2))
cat("\nValeurs singulières :", round(sv2, 4), "\n")
```

```
##
## Valeurs singulières : 9.7685 2.6881 1.2829 0.8396 0
```

```r
cat("Rang de A :", rang2, "\n")
```

```
## Rang de A : 4
```

```r
Vr2 <- V2[, 1:rang2]
Ur2 <- matrix(0, nrow(A2), rang2)
for (i in 1:rang2) {
  Av <- A2 %*% Vr2[, i]
  Ur2[, i] <- Av / sqrt(sum(Av^2))
}

D2_inv <- diag(1/sv2[1:rang2])

# Pseudo-inverse : A+ = Vr D^{-1} Ur^T
A2_plus <- Vr2 %*% D2_inv %*% t(Ur2)
cat("\nPseudo-inverse A  :\n"); print(round(A2_plus, 6))
```

```
##
## Pseudo-inverse A  :

##       [,1]  [,2]  [,3] [,4]
## [1,] -0.05 -0.35  0.65  0.0
## [2,] -0.05 -0.35  0.65  0.0
## [3,]  0.00  0.00 -0.50  0.5
## [4,]  0.10 -0.30  0.20  0.5
## [5,]  0.10 -0.30 -0.30  0.0
```

```
cat("\nVérification Moore-Penrose :\n")
```

```
##
## Vérification Moore-Penrose :
```

```
cat("  ||A A  A - A|| =", round(norm(A2 %*% A2_plus %*% A2 - A2, "F"), 10), "\n")
```

```
##   ||A A  A - A|| = 0
```

```
cat("  ||A  A A  - A || =", round(norm(A2_plus %*% A2 %*% A2_plus - A2_plus, "F"), 10), "\n")
```

```
##   ||A  A A  - A || = 0
```

```
AA_p <- A2 %*% A2_plus
cat("  ||(AA )  - AA || =", round(norm(t(AA_p) - AA_p, "F"), 10), "\n")
```

```
##   ||(AA )  - AA || = 0
```

```
Ap_A <- A2_plus %*% A2
cat("  ||(A A)  - A A|| =", round(norm(t(Ap_A) - Ap_A, "F"), 10), "\n")
```

```
##   ||(A A)  - A A|| = 0
```

4.2.2 Pseudo-inverse de B

```
cat("===== PSEUDO-INVERSE DE B (4×5) =====\n\n")
```

```
## ===== PSEUDO-INVERSE DE B (4×5) =====
```

```
B2 <- matrix(c( 4,  0, -1, -2,  0,
               -5,  0,  3,  5,  0,
                2,  0, -1, -2,  0,
                6,  0, -3, -6,  0), nrow = 4, byrow = TRUE)
cat("Matrice B :\n"); print(B2)
```

```
## Matrice B :
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    4    0   -1   -2    0
## [2,]   -5    0    3    5    0
## [3,]    2    0   -1   -2    0
## [4,]    6    0   -3   -6    0
```

```r
BtB2 <- t(B2) %*% B2
eigB2 <- eigen(BtB2)
svB2 <- sqrt(abs(eigB2$values))
ordB2 <- order(svB2, decreasing = TRUE)
svB2 <- svB2[ordB2]
VB2 <- eigB2$vectors[, ordB2]

rangB2 <- min(sum(svB2 > 1e-10), nrow(B2), ncol(B2))
cat("\nValeurs singulières :", round(svB2, 4), "\n")
```

```
##
## Valeurs singulières : 12.9536 1.4455 0.3378 0 0
```

```r
cat("Rang de B :", rangB2, "\n")
```

```
## Rang de B : 4
```

```r
VrB2 <- VB2[, 1:rangB2]
UrB2 <- matrix(0, nrow(B2), rangB2)
for (i in 1:rangB2) {
  Bv <- B2 %*% VrB2[, i]
  UrB2[, i] <- Bv / sqrt(sum(Bv^2))
}

if (rangB2 == 1) {
  DB2_inv <- matrix(1/svB2[1], 1, 1)
} else {
  DB2_inv <- diag(1/svB2[1:rangB2])
}

B2_plus <- VrB2 %*% DB2_inv %*% t(UrB2)
cat("\nPseudo-inverse B  :\n"); print(round(B2_plus, 6))
```

```
##
## Pseudo-inverse B  :
```

```
##            [,1]          [,2]        [,3]          [,4]
## [1,]        0.5  0.000000e+00       -0.05 -1.500000e-01
## [2,] -7577261.9 -3.099789e+07 -7577261.86 -2.273179e+07
## [3,]        0.0  2.000001e+00        0.50  1.500001e+00
## [4,]        0.5 -1.000000e+00       -0.35 -1.050000e+00
## [5,]        0.0  0.000000e+00        0.00  0.000000e+00
```

```r
cat("\nVérification : ||B B  B - B|| =", round(norm(B2 %*% B2_plus %*% B2 - B2, "F"), 10), "\n")
```

```
##
## Vérification : ||B B  B - B|| = 2.976e-07
```

4.2.3 Résolution de Ax = b et Bx = b

```r
b_vec <- c(6, -1, -4, 6)
cat("===== Résolution Ax = b, b = (6, -1, -4, 6) =====\n\n")
```

```
## ===== Résolution Ax = b, b = (6, -1, -4, 6) =====
```

```r
x_A <- A2_plus %*% b_vec
cat("Solution x = A b :\n"); print(round(x_A, 6))
```

```
## Solution x = A b :
```

```
##       [,1]
## [1,] -2.55
## [2,] -2.55
## [3,]  5.00
## [4,]  3.10
## [5,]  2.10
```

```r
Ax <- A2 %*% x_A
residu_A <- norm(Ax - b_vec, "2")
cat("||Ax - b|| =", round(residu_A, 10), "\n")
```

```
## ||Ax - b|| = 0
```

```r
if (residu_A < 1e-8) {
  cat("=> Système COMPATIBLE\n")
} else {
  cat("=> Système INCOMPATIBLE, solution au sens des moindres carrés\n")
}
```

```
## => Système COMPATIBLE
```

```r
cat("Dim espace nul :", ncol(A2) - rang2, "\n\n")
```

```
## Dim espace nul : 1
```

```r
cat("===== Résolution Bx = b =====\n\n")
```

```
## ===== Résolution Bx = b =====
```

```r
x_B <- B2_plus %*% b_vec
cat("Solution x = B b :\n"); print(round(x_B, 6))
```

```
## Solution x = B b :

##               [,1]
## [1,]  2.300000e+00
## [2,] -1.205473e+08
## [3,]  5.000003e+00
## [4,] -9.000020e-01
## [5,]  0.000000e+00
```

```r
Bx <- B2 %*% x_B
residu_B <- norm(Bx - b_vec, "2")
cat("||Bx - b|| =", round(residu_B, 10), "\n")
```

```
## ||Bx - b|| = 5.6921
```

```r
if (residu_B < 1e-8) {
  cat("=> Système COMPATIBLE\n")
} else {
  cat("=> Système INCOMPATIBLE, solution au sens des moindres carrés\n")
}
```

```
## => Système INCOMPATIBLE, solution au sens des moindres carrés
```

```r
cat("Dim espace nul :", ncol(B2) - rangB2, "\n")
```

```
## Dim espace nul : 1
```

5 Exercice 5 — ACP (Analyse en Composantes Principales)

5.1 Partie 1 : Première approche

5.1.1 Chargement des données

```
donnees1 <- read.table("donnees1.txt", header = TRUE, row.names = 1)
cat("Dimensions :", nrow(donnees1), "pays ×", ncol(donnees1), "variables\n\n")
```

```
## Dimensions : 25 pays × 10 variables
```

```
knitr::kable(donnees1, caption = "Données de consommation protéique en Europe")
```

Table 2: Données de consommation protéique en Europe

	VROU	VBLA	OEUF	LAIT	POISS	CERE	AMID	NOIX	FRLEG	EST
al	10.1	1.4	0.5	8.9	0.2	42.3	0.6	5.5	1.7	1
at	8.9	14.0	4.3	19.9	2.1	28.0	3.6	1.3	4.3	0
be	13.5	9.3	4.1	17.5	4.5	26.6	5.7	2.1	4.0	0
bg	7.8	6.0	1.6	8.3	1.2	56.7	1.1	3.7	4.2	1
ch	13.1	10.1	3.1	23.8	2.3	25.6	2.8	2.4	2.4	0
cz	9.7	11.4	2.8	12.5	2.0	34.3	5.0	1.1	4.0	1
dd	8.4	11.6	3.7	11.1	5.4	24.6	6.5	0.8	3.6	1
de	13.4	10.8	3.7	18.8	3.4	18.6	5.2	1.5	3.8	0
dk	10.6	10.8	3.7	25.0	9.9	21.9	4.8	0.7	2.4	0
es	7.1	3.4	3.1	8.6	7.0	29.2	5.9	5.7	7.9	0
fi	9.5	4.7	2.7	33.7	5.8	26.3	5.1	1.0	1.4	0
fr	18.0	9.9	3.3	19.5	5.7	28.1	4.8	2.4	6.5	0
gr	10.2	3.0	2.8	17.6	5.9	41.7	2.2	7.8	6.5	0
hu	5.3	12.4	2.9	9.7	0.3	40.1	4.0	5.4	4.2	1
ie	13.9	10.0	4.7	25.8	2.2	24.0	6.2	1.6	2.9	0
it	9.0	5.1	2.9	13.7	3.4	36.8	2.1	4.3	6.7	0
nl	9.5	13.6	3.6	23.4	2.5	22.4	4.2	1.8	3.7	0
no	9.4	4.7	2.7	23.3	9.7	23.0	4.6	1.6	2.7	0
pl	6.9	10.2	2.7	19.3	3.0	36.1	5.9	2.0	6.6	1
pt	6.2	3.7	1.1	4.9	14.2	27.0	5.9	4.7	7.9	0
ro	6.2	6.3	1.5	11.1	1.0	49.6	3.1	5.3	2.8	1
ru	9.3	4.6	2.1	16.6	3.0	43.6	6.4	3.4	2.9	1
se	9.9	7.8	3.5	24.7	7.5	19.5	3.7	1.4	2.0	0
uk	17.4	5.7	4.7	20.6	4.3	24.3	4.7	3.4	3.3	0
yu	4.4	5.0	1.2	9.5	0.6	55.9	3.0	5.7	3.2	1

5.1.2 Question 1 : Matrice de corrélation

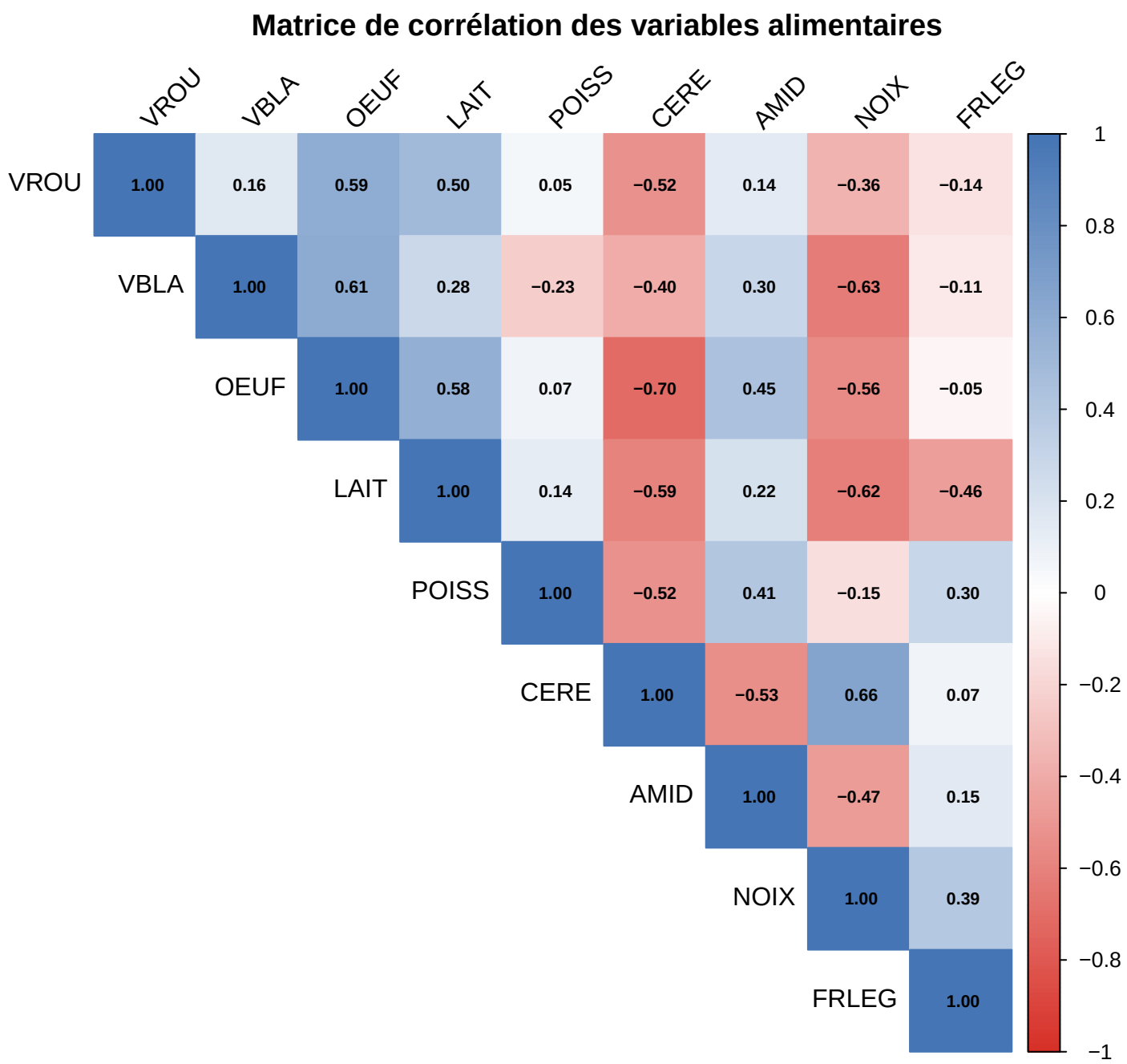
```
vars_quant <- donnees1[, 1:9]
cor_matrix <- cor(vars_quant)
cat("Matrice de corrélation :\n")
```

```
## Matrice de corrélation :
```

```r
print(round(cor_matrix, 2))
```

```
##        VROU  VBLA  OEUF  LAIT POISS  CERE  AMID  NOIX FRLEG
## VROU   1.00  0.16  0.59  0.50  0.05 -0.52  0.14 -0.36 -0.14
## VBLA   0.16  1.00  0.61  0.28 -0.23 -0.40  0.30 -0.63 -0.11
## OEUF   0.59  0.61  1.00  0.58  0.07 -0.70  0.45 -0.56 -0.05
## LAIT   0.50  0.28  0.58  1.00  0.14 -0.59  0.22 -0.62 -0.46
## POISS  0.05 -0.23  0.07  0.14  1.00 -0.52  0.41 -0.15  0.30
## CERE  -0.52 -0.40 -0.70 -0.59 -0.52  1.00 -0.53  0.66  0.07
## AMID   0.14  0.30  0.45  0.22  0.41 -0.53  1.00 -0.47  0.15
## NOIX  -0.36 -0.63 -0.56 -0.62 -0.15  0.66 -0.47  1.00  0.39
## FRLEG -0.14 -0.11 -0.05 -0.46  0.30  0.07  0.15  0.39  1.00
```

```r
library(corrplot)
corrplot(cor_matrix,
         method = "color",
         col = colorRampPalette(c("#D73027", "#FFFFFF", "#4575B4"))(200),
         type = "upper",
         tl.col = "black", tl.srt = 45,
         addCoef.col = "black", number.cex = 0.7,
         title = "Matrice de corrélation des variables alimentaires",
         mar = c(0, 0, 2, 0))
```



5.1.3 Question 2 : Variables corrélées / décorrélées

```
cat("=== Corrélations positives fortes ===\n")
```

```
## === Corrélations positives fortes ===
```

```
cat("  VBLA - OEUF :", round(cor_matrix["VBLA", "OEUF"], 3), "\n")
```

```
##   VBLA - OEUF : 0.608
```

```r
cat("  VROU - OEUF :", round(cor_matrix["VROU", "OEUF"], 3), "\n")
```

```
##   VROU - OEUF : 0.593
```

```r
cat("  LAIT - OEUF :", round(cor_matrix["LAIT", "OEUF"], 3), "\n\n")
```

```
##   LAIT - OEUF : 0.58
```

```r
cat("=== Corrélations négatives fortes ===\n")
```

```
## === Corrélations négatives fortes ===
```

```r
cat("  OEUF - CERE :", round(cor_matrix["OEUF", "CERE"], 3), "\n")
```

```
##   OEUF - CERE : -0.703
```

```r
cat("  LAIT - CERE :", round(cor_matrix["LAIT", "CERE"], 3), "\n\n")
```

```
##   LAIT - CERE : -0.593
```

```r
cat("=== Variables faiblement corrélées ===\n")
```

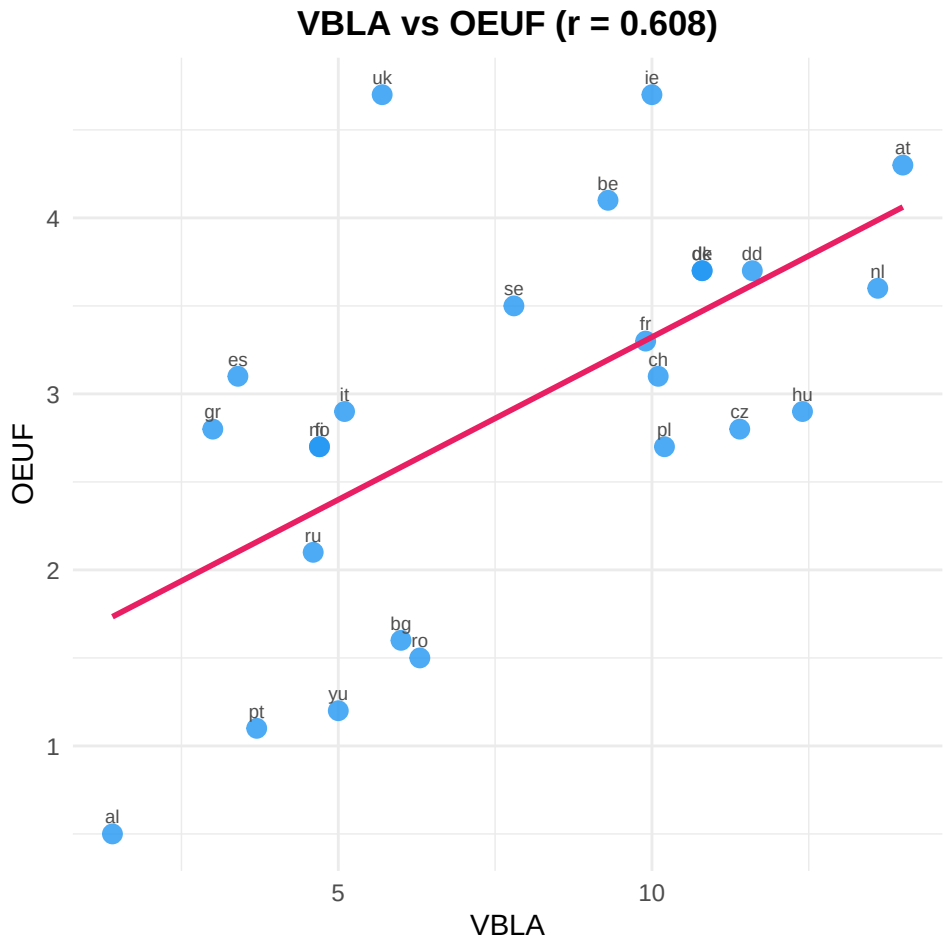
```
## === Variables faiblement corrélées ===
```

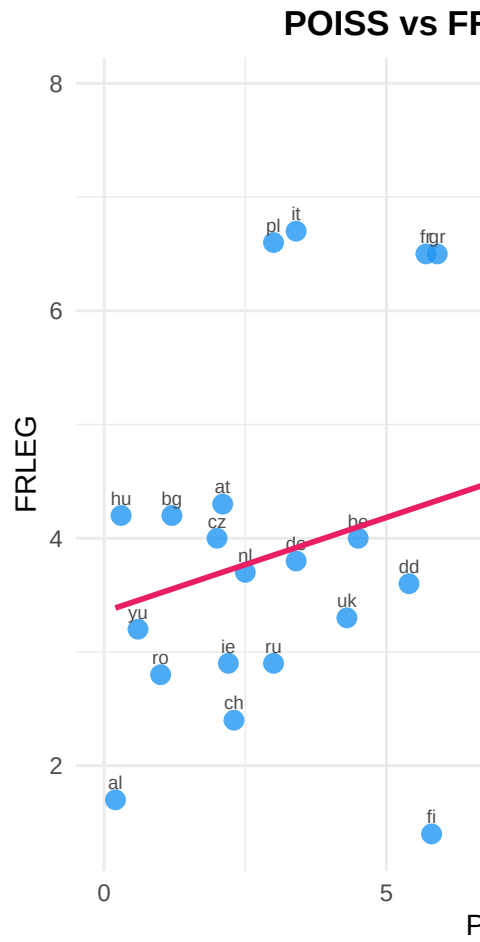
```r
cat("  FRLEG est relativement indépendante des autres variables.\n")
```

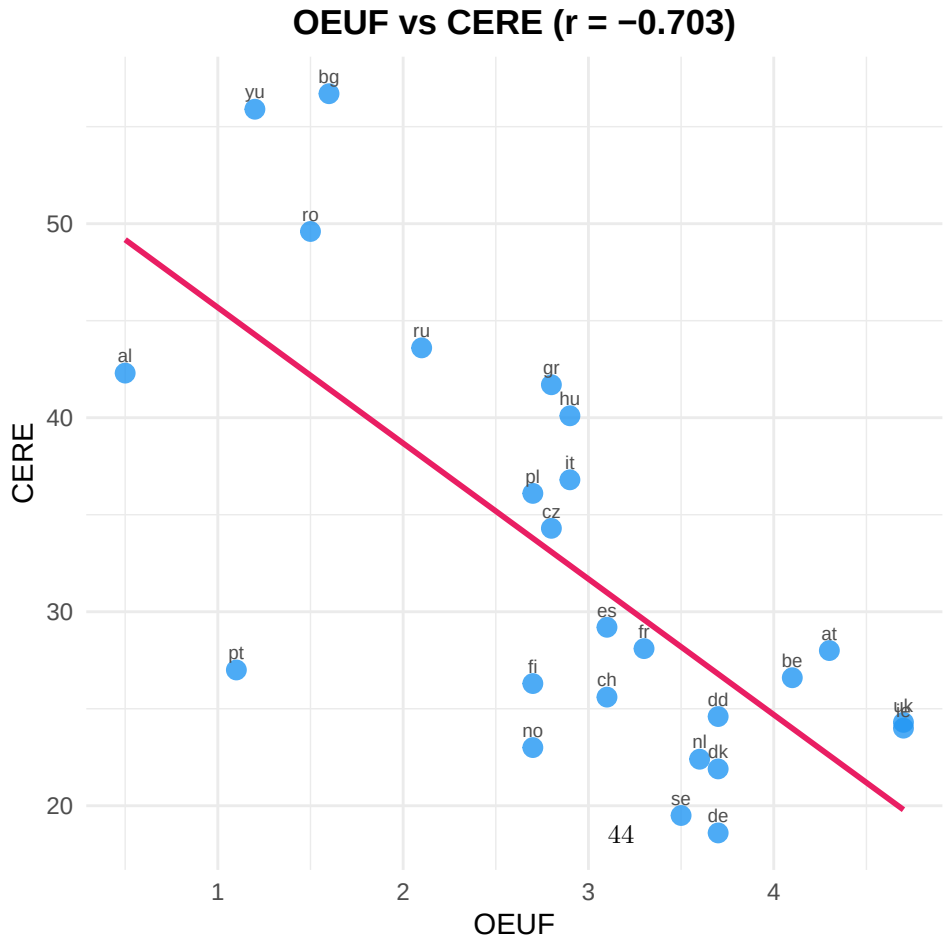
```
##   FRLEG est relativement indépendante des autres variables.
```

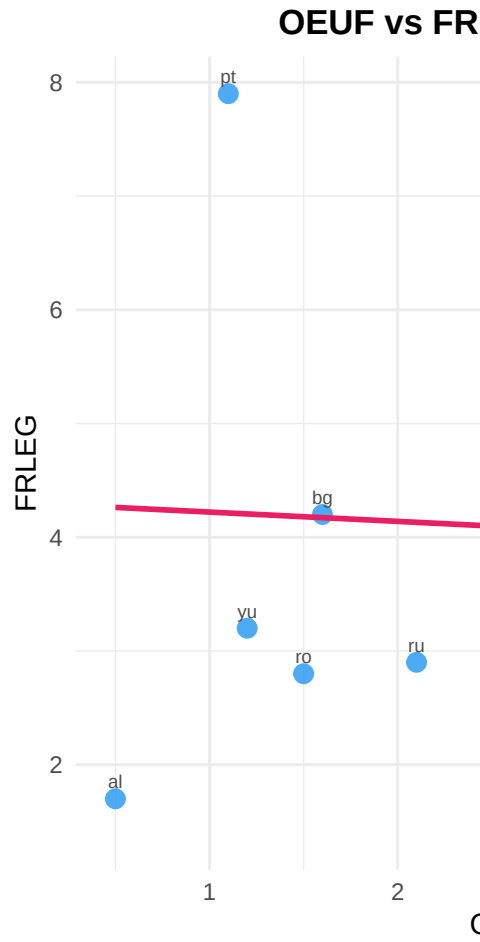
5.1.4 Question 3 : Graphiques de couples

```r
library(ggplot2)
library(gridExtra)

plot_couple <- function(data, xvar, yvar, titre) {
  df <- data.frame(x = data[, xvar], y = data[, yvar], pays = rownames(data))
  r <- round(cor(df$x, df$y), 3)
  ggplot(df, aes(x = x, y = y)) +
    geom_point(color = "#2196F3", size = 3, alpha = 0.8) +
    geom_text(aes(label = pays), size = 2.5, vjust = -0.8, color = "gray30") +
    geom_smooth(method = "lm", color = "#E91E63", se = FALSE, linewidth = 1) +
    labs(title = paste0(titre, " (r = ", r, ")"), x = xvar, y = yvar) +
    theme_minimal() +
    theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 13))
}

p1 <- plot_couple(donnees1, "VBLA", "OEUF", "VBLA vs OEUF")
p2 <- plot_couple(donnees1, "POISS", "FRLEG", "POISS vs FRLEG")
p3 <- plot_couple(donnees1, "OEUF", "CERE", "OEUF vs CERE")
p4 <- plot_couple(donnees1, "OEUF", "FRLEG", "OEUF vs FRLEG")

grid.arrange(p1, p2, p3, p4, ncol = 2)
```


VBLA vs OEUF (r = 0.608)
OEUF
VBLA
uk
ie
at
be
dd
nl
se
fr
ch
es
it
gr
hu
cz
pl
ru
bg
ro
yu
pt
al
1
2
3
4
5
10


POISS vs FR
FRLEG
pl
it
hu
bg
at
cz
nl
de
dd
uk
yu
ro
ie
ru
ch
al
fi
2
4
6
8
0
5


OEUF vs CERE (r = −0.703)
CERE
OEUF
yu
bg
ro
ru
al
gr
hu
pl
it
cz
es
fr
pt
fi
ch
at
be
dd
no
nl
dk
se
de
20
30
40
50
1
2
3
4


OEUF vs FR
FRLEG
pt
bg
yu
ro
ru
al
2
4
6
8
1
2

Commentaires : VBLA vs OEUF montre une corrélation positive nette. OEUF vs CERE une forte corrélation négative (opposition produits animaux/céréales). POISS vs FRLEG et OEUF vs FRLEG sont faiblement corrélés. Le Portugal (pt) est atypique pour le poisson.

5.2 Partie 2 : ACP centrée-réduite

5.2.1 Question 4 : Résultats de l'ACP

```r
library(FactoMineR)
library(factoextra)

acp <- PCA(donnees1, scale.unit = TRUE, quali.sup = 10, ncp = 5, graph = FALSE)

cat("===== Valeurs propres =====\n")
```

```
## ===== Valeurs propres =====
```

```r
knitr::kable(round(acp$eig, 2),
             col.names = c("Val. propre", "% variance", "% cumulé"),
             caption = "Valeurs propres et variance expliquée")
```

Table 3: Valeurs propres et variance expliquée

	Val. propre	% variance	% cumulé
comp 1	4.01	44.59	44.59
comp 2	1.71	19.04	63.64
comp 3	1.13	12.57	76.20
comp 4	0.90	9.97	86.17
comp 5	0.44	4.93	91.09
comp 6	0.34	3.77	94.86
comp 7	0.24	2.63	97.49
comp 8	0.12	1.30	98.79
comp 9	0.11	1.21	100.00

```r
cat("===== Corrélations variables / axes =====\n")
```

```
## ===== Corrélations variables / axes =====
```

```r
knitr::kable(round(acp$var$coord, 4), caption = "Corrélations des variables")
```

Table 4: Corrélations des variables

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
VROU	0.6228	-0.1040	-0.3164	0.6167	0.1889
VBLA	0.6118	-0.2771	0.6751	-0.0405	-0.2242
OEUF	0.8495	-0.0168	0.1967	0.3361	-0.0086
LAIT	0.7612	-0.2546	-0.3927	-0.0556	-0.0779

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
POISS	0.2654	0.8093	-0.3761	-0.2173	-0.2144
CERE	-0.8746	-0.2925	0.1330	-0.0016	0.1665
AMID	0.5802	0.5069	0.2711	-0.2621	0.5051
NOIX	-0.8441	0.1594	-0.0845	0.3438	0.0482
FRLEG	-0.2560	0.7337	0.3773	0.4060	-0.1417

```r
cat("===== Valeurs test de EST =====\n")
```

```
## ===== Valeurs test de EST =====
```

```r
print(round(acp$quali.sup$v.test[, 1:5], 2))
```

```
##   Dim.1 Dim.2 Dim.3 Dim.4 Dim.5
## 0  2.77  1.68 -1.89  1.67 -1.65
## 1 -2.77 -1.68  1.89 -1.67  1.65
```

```r
cat("===== Coordonnées des individus =====\n")
```

```
## ===== Coordonnées des individus =====
```

```r
knitr::kable(round(acp$ind$coord, 2), caption = "Coordonnées des individus")
```

Table 5: Coordonnées des individus

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
al	-3.53	-1.78	-1.73	0.21	0.05
at	1.46	-0.96	1.47	0.15	-1.04
be	1.65	0.22	0.29	0.61	0.63
bg	-3.19	-1.34	0.21	0.16	-0.47
ch	1.11	-1.53	-0.60	0.48	-0.56
cz	0.38	-0.51	1.33	-0.43	0.26
dd	1.44	0.59	1.41	-1.02	0.33
de	2.02	-0.16	0.36	0.44	0.15
dk	2.42	0.30	-0.67	-1.02	-0.80
es	-1.35	2.85	0.50	0.56	0.41
fi	1.60	-0.60	-1.97	-1.50	0.31
fr	1.50	0.78	-0.03	1.91	0.15
gr	-2.30	0.91	-1.05	1.77	-0.55
hu	-1.49	-0.74	2.02	-0.11	-0.20
ie	2.73	-0.69	0.14	0.53	0.96
it	-1.58	0.37	0.03	1.16	-0.86
nl	1.68	-0.85	0.89	-0.16	-0.80
no	1.00	0.79	-1.71	-1.20	-0.34
pl	0.10	0.65	1.51	-0.51	0.13
pt	-1.80	4.30	-0.26	-0.93	-0.40
ro	-2.81	-1.13	0.16	-0.57	0.34
ru	-0.81	-0.07	-0.27	-0.83	1.80
se	1.68	-0.23	-1.22	-0.79	-0.84

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
uk	1.78	-0.11	-1.12	1.86	0.87
yu	-3.70	-1.05	0.28	-0.78	0.45

```r
cat("===== Qualité de représentation (cos²) =====\n")
```

```
## ===== Qualité de représentation (cos²) =====
```

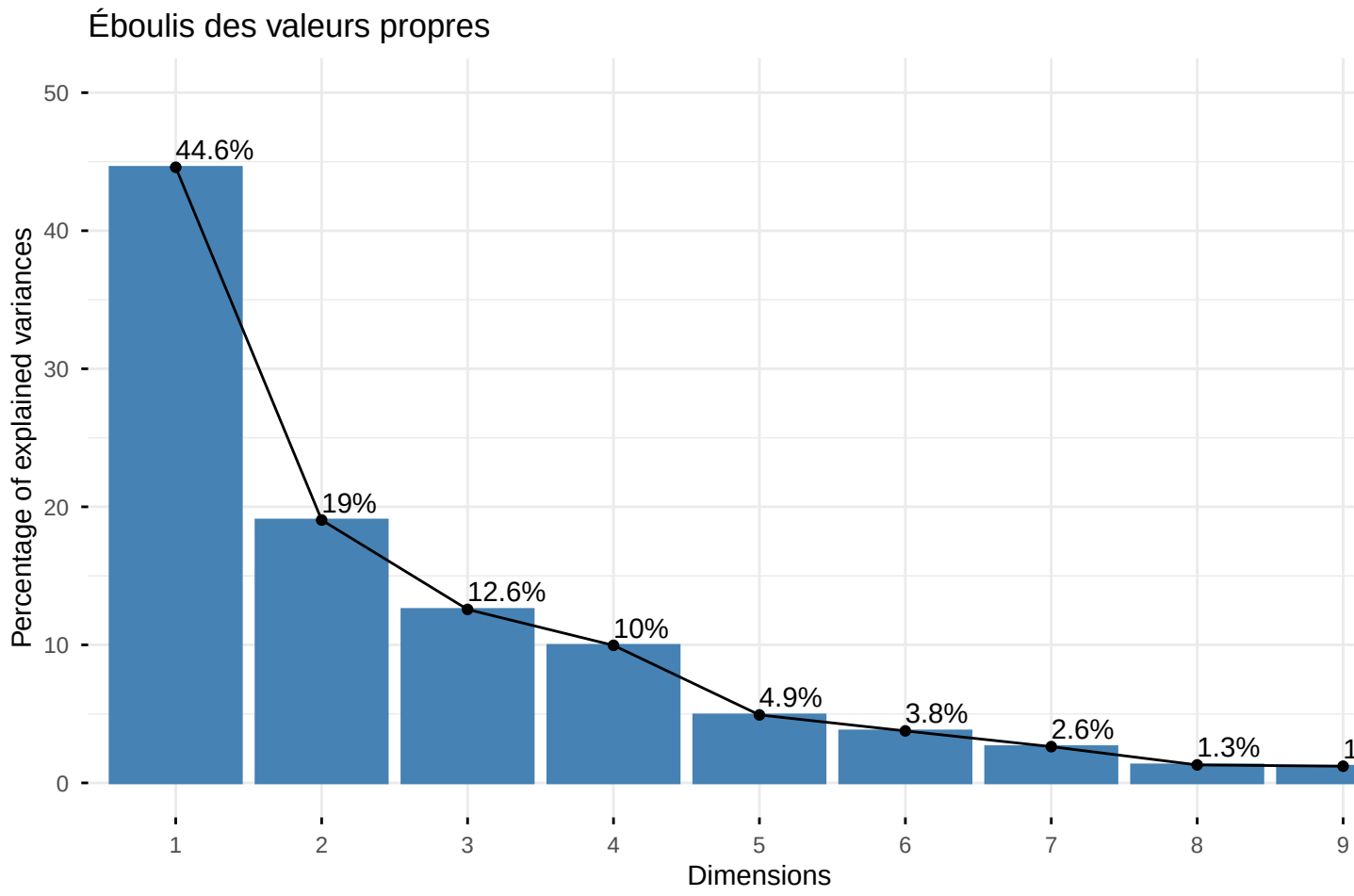
```r
knitr::kable(round(acp$ind$cos2, 4), caption = "cos² des individus")
```

Table 6: cos² des individus

	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
al	0.6081	0.1547	0.1459	0.0022	0.0001
at	0.3275	0.1411	0.3331	0.0035	0.1653
be	0.7134	0.0126	0.0218	0.0982	0.1040
bg	0.7358	0.1309	0.0032	0.0018	0.0160
ch	0.2581	0.4914	0.0740	0.0479	0.0643
cz	0.0442	0.0827	0.5519	0.0587	0.0214
dd	0.3172	0.0523	0.3033	0.1595	0.0170
de	0.8088	0.0048	0.0255	0.0378	0.0044
dk	0.6732	0.0105	0.0517	0.1181	0.0737
es	0.1562	0.6950	0.0217	0.0266	0.0141
fi	0.2426	0.0346	0.3684	0.2155	0.0094
fr	0.2590	0.0702	0.0001	0.4184	0.0026
gr	0.4315	0.0670	0.0900	0.2555	0.0244
hu	0.2738	0.0668	0.4991	0.0014	0.0047
ie	0.7881	0.0501	0.0022	0.0301	0.0971
it	0.4572	0.0256	0.0002	0.2465	0.1354
nl	0.5183	0.1337	0.1465	0.0048	0.1164
no	0.1613	0.1023	0.4756	0.2349	0.0183
pl	0.0019	0.0854	0.4662	0.0526	0.0034
pt	0.1377	0.7831	0.0028	0.0366	0.0066
ro	0.8043	0.1311	0.0027	0.0328	0.0117
ru	0.1339	0.0009	0.0148	0.1417	0.6706
se	0.4784	0.0088	0.2493	0.1039	0.1189
uk	0.3465	0.0014	0.1357	0.3756	0.0821
yu	0.8626	0.0693	0.0051	0.0379	0.0126

5.2.2 Question 5 : Répartition de l'inertie

```r
fviz_eig(acp, addlabels = TRUE, ylim = c(0, 50),
         main = "Éboulis des valeurs propres")
```




```r
cat("2 premiers axes :", round(acp$eig[2, 3], 2), "% d'inertie\n")
```

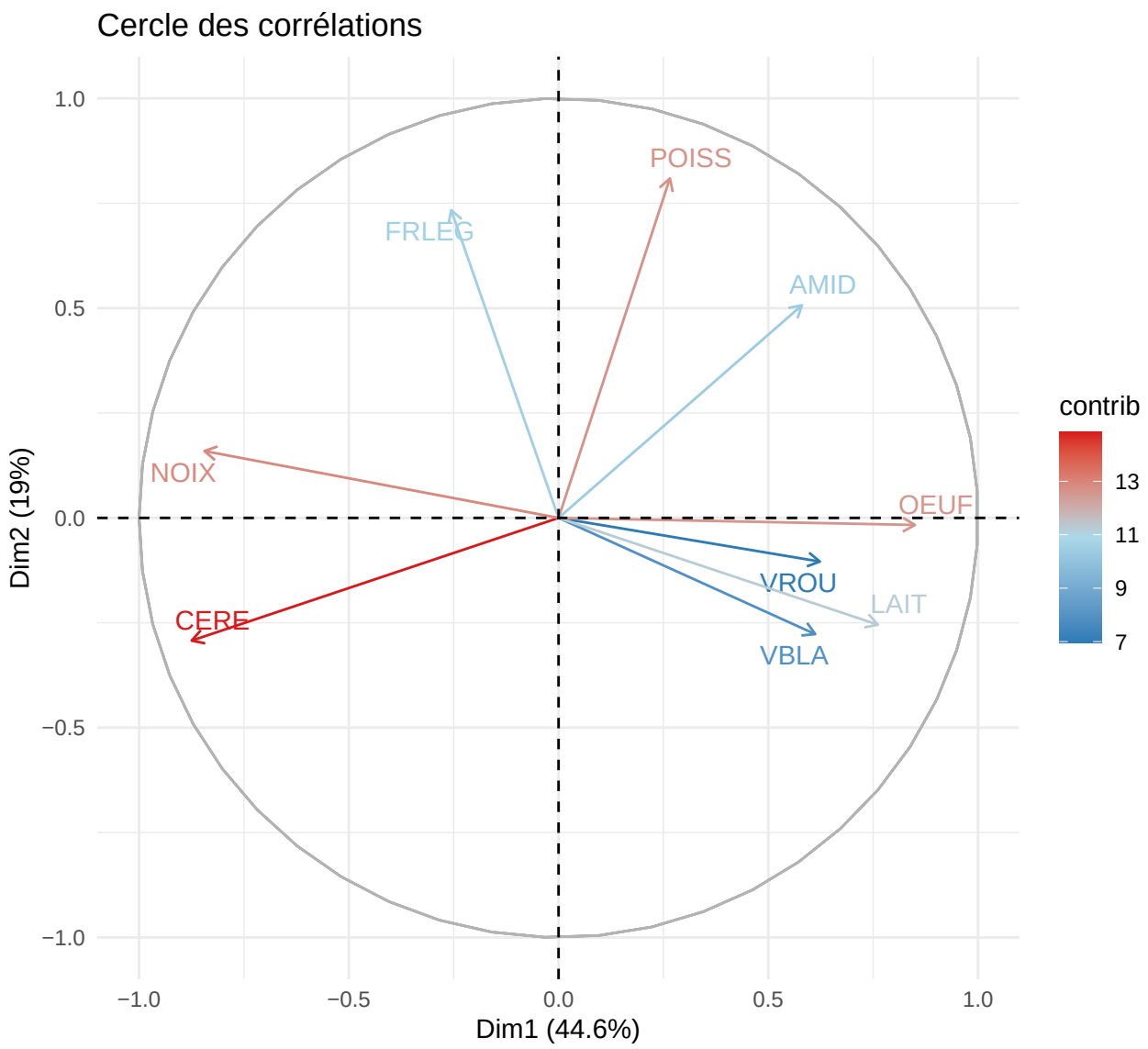
```
## 2 premiers axes : 63.64 % d'inertie
```

```r
cat("On retient 2 axes (voire 3 pour ~75%).\n")
```

```
## On retient 2 axes (voire 3 pour ~75%).
```

5.2.3 Question 6 : Aliments déterminant les axes

```r
fviz_pca_var(acp, col.var = "contrib",
             gradient.cols = c("#2c7bb6", "#abd9e9", "#d7191c"),
             repel = TRUE, title = "Cercle des corrélations") + theme_minimal()
```




```r
cat("Axe 1 : OEUF, LAIT, VBLA (+) vs CERE, NOIX (-)\n")
```

```
## Axe 1 : OEUF, LAIT, VBLA (+) vs CERE, NOIX (-)
```

```r
cat("  => Opposition produits animaux / céréales\n")
```

```
##   => Opposition produits animaux / céréales
```

```r
cat("Axe 2 : POISS, FRLEG (+) vs AMID (-)\n")
```

```
## Axe 2 : POISS, FRLEG (+) vs AMID (-)
```

```r
cat("  => Régime méditerranéen\n")
```

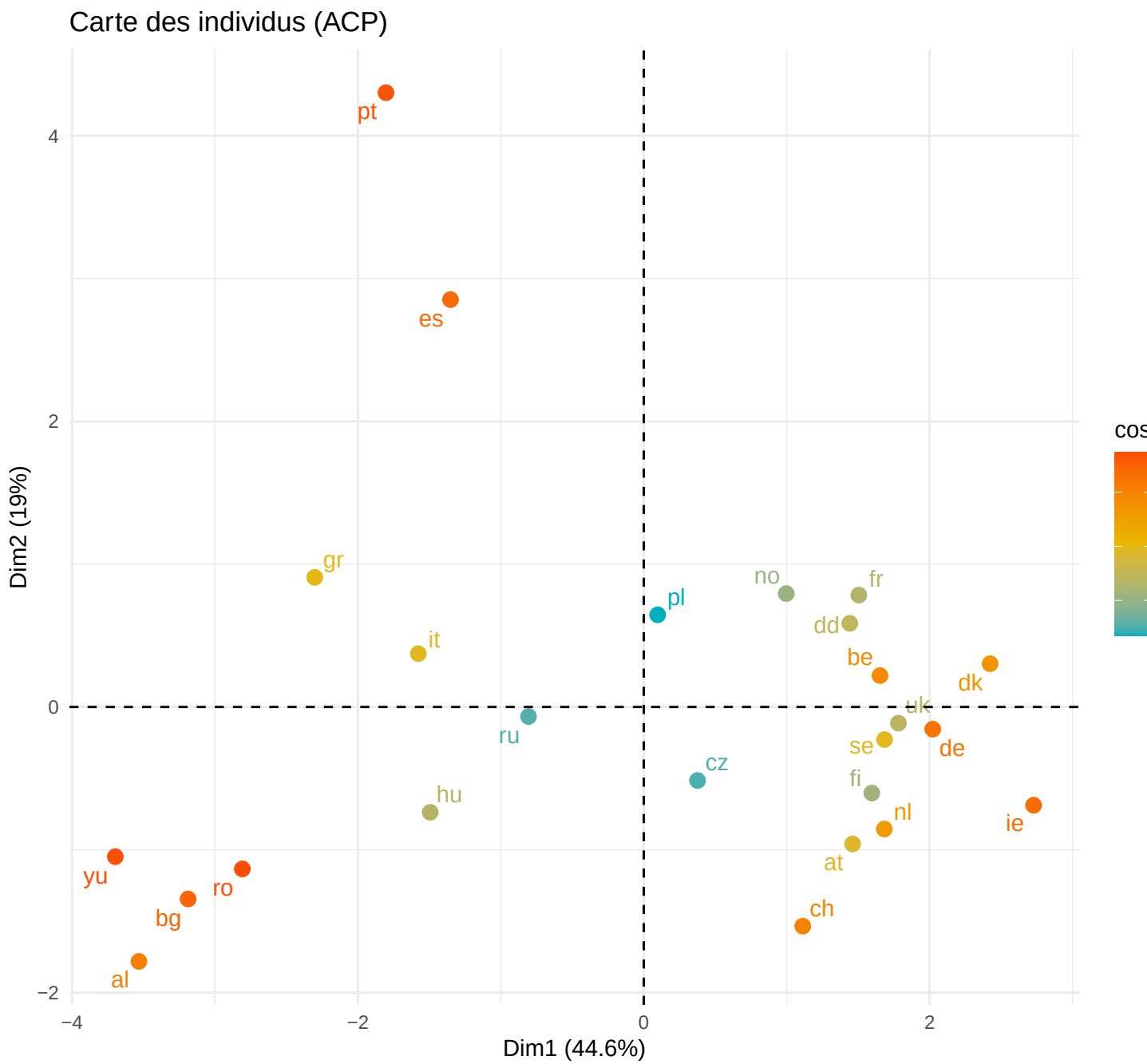
```
##   => Régime méditerranéen
```

```r
cat("Effet de taille : limité (données centrées-réduites)\n")
```

```
## Effet de taille : limité (données centrées-réduites)
```

5.2.4 Question 7 : Pays déterminant les axes

```r
fviz_pca_ind(acp, col.ind = "cos2",
             gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
             repel = TRUE, pointsize = 3,
             title = "Carte des individus (ACP)") + theme_minimal()
```




```r
cat("Axe 1 : bg, ro, yu, al (Est, négatif) vs ie, dk, nl, de (Ouest, positif)\n")
```

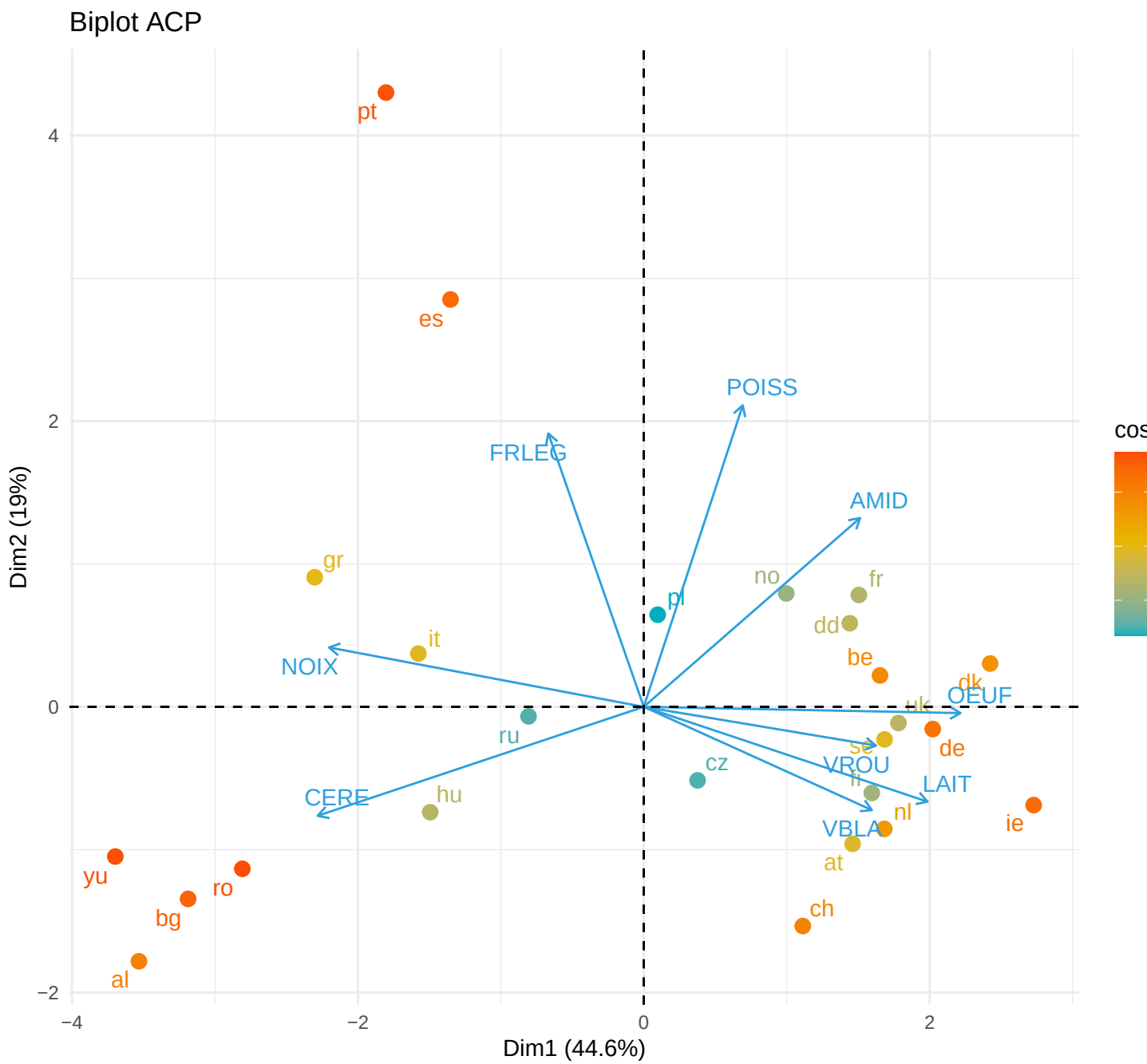
```
## Axe 1 : bg, ro, yu, al (Est, négatif) vs ie, dk, nl, de (Ouest, positif)
```

```r
cat("Axe 2 : pt, es, gr (méditerranéens, positif)\n")
```

```
## Axe 2 : pt, es, gr (méditerranéens, positif)
```

5.2.5 Question 8 : Interprétation des axes

```
fviz_pca_biplot(acp, col.ind = "cos2", col.var = "#2E9FDF",
                gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
                repel = TRUE, pointsize = 3,
                title = "Biplot ACP") + theme_minimal()
```



```r
cat("Axe 1 : Opposition Est/Ouest (produits animaux vs céréales)\n")
```

```
## Axe 1 : Opposition Est/Ouest (produits animaux vs céréales)
```

```r
cat("Axe 2 : Gradient méditerranéen (poisson, fruits/légumes)\n")
```

```
## Axe 2 : Gradient méditerranéen (poisson, fruits/légumes)
```

5.2.6 Question 9 : Cinq pays mal représentés

```r
cos2_plan <- acp$ind$cos2[, 1] + acp$ind$cos2[, 2]
cos2_df <- data.frame(Pays = names(cos2_plan), Cos2 = round(cos2_plan, 4))
cos2_df <- cos2_df[order(cos2_df$Cos2), ]
cat("5 pays les moins bien représentés (somme cos² axes 1+2) :\n")
```

```
## 5 pays les moins bien représentés (somme cos² axes 1+2) :
```

```r
knitr::kable(head(cos2_df, 5), row.names = FALSE)
```

Pays	Cos2
pl	0.0873
cz	0.1269
ru	0.1349
no	0.2636
fi	0.2772

5.2.7 Question 10 : Cas du Portugal

```r
cat("Coordonnées du Portugal :\n")
```

```
## Coordonnées du Portugal :
```

```r
print(round(acp$ind$coord["pt", ], 4))
```

```
##   Dim.1   Dim.2   Dim.3   Dim.4   Dim.5
## -1.8035  4.3007 -0.2562 -0.9295 -0.3963
```

```r
cat("\ncos² :\n")
```

```
##
## cos² :
```

```r
print(round(acp$ind$cos2["pt", ], 4))
```

```
##  Dim.1  Dim.2  Dim.3  Dim.4  Dim.5
## 0.1377 0.7831 0.0028 0.0366 0.0066
```

```r
cat("\nContributions (%) :\n")
```

```
##
## Contributions (%) :
```

```r
print(round(acp$ind$contrib["pt", ], 2))
```

```
## Dim.1 Dim.2 Dim.3 Dim.4 Dim.5
##  3.24 43.17  0.23  3.85  1.42
```

```r
cat("\nLe Portugal est très bien représenté sur l'axe 2 (poisson élevé).\n")
```

```
##
## Le Portugal est très bien représenté sur l'axe 2 (poisson élevé).
```

```r
cat("Il contribue fortement à l'axe 2. Individu atypique mais réel.\n")
```

```
## Il contribue fortement à l'axe 2. Individu atypique mais réel.
```

```r
cat("On pourrait le traiter en individu supplémentaire pour vérifier\n")
```

```
## On pourrait le traiter en individu supplémentaire pour vérifier
```

```r
cat("qu'il ne déforme pas les résultats.\n")
```

```
## qu'il ne déforme pas les résultats.
```

5.3 Partie 3 : Variable supplémentaire EST

5.3.1 Question 11 : Données suffisantes ?

```r
cat("n total =", nrow(donnees1), ", n_Est =", sum(donnees1$EST==1),
    ", n_Ouest =", sum(donnees1$EST==0), "\n")
```

```
## n total = 25 , n_Est = 9 , n_Ouest = 16
```

```r
cat("L'effectif est suffisant pour calculer les valeurs test,\n")
```

```
## L'effectif est suffisant pour calculer les valeurs test,
```

```r
cat("mais l'approximation normale est à prendre avec prudence (n=25).\n")
```

```
## mais l'approximation normale est à prendre avec prudence (n=25).
```

5.3.2 Question 12 : Conditions et interprétation

```r
vt <- acp$quali.sup$v.test
print(round(vt[, 1:5], 2))
```

```
##   Dim.1 Dim.2 Dim.3 Dim.4 Dim.5
## 0  2.77  1.68 -1.89  1.67 -1.65
## 1 -2.77 -1.68  1.89 -1.67  1.65
```

```r
cat("\nUne v.test > |1.96| est significative à 5%.\n")
```

```
##
## Une v.test > |1.96| est significative à 5%.
```

```r
cat("L'axe 1 sépare bien les pays de l'Est (valeurs négatives, céréales)\n")
```

```
## L'axe 1 sépare bien les pays de l'Est (valeurs négatives, céréales)
```

```r
cat("des pays occidentaux. Les conditions de normalité ne sont pas\n")
```

```
## des pays occidentaux. Les conditions de normalité ne sont pas
```

```r
cat("parfaitement réunies (petit échantillon), mais les résultats\n")
```

```
## parfaitement réunies (petit échantillon), mais les résultats
```

```r
cat("sont cohérents avec l'interprétation géographique.\n")
```

```
## sont cohérents avec l'interprétation géographique.
```

6 Exercice 6 — Test du Chi-deux

```r
cat("===== TEST DU CHI-DEUX D'ADÉQUATION =====\n\n")
```

```
## ===== TEST DU CHI-DEUX D'ADÉQUATION =====
```

```r
jours <- c("Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi")
observes <- c(30, 14, 34, 45, 57, 20)
prob_theo <- c(0.10, 0.10, 0.15, 0.20, 0.30, 0.15)

n <- sum(observes)
theoriques <- n * prob_theo
k <- length(jours)

cat("n =", n, "clients\n\n")
```

```
## n = 200 clients
```

```r
knitr::kable(data.frame(Jour = jours, Oi = observes, pi = prob_theo, Ei = theoriques),
             caption = "Effectifs observés et théoriques")
```

Table 8: Effectifs observés et théoriques

Jour	Oi	pi	Ei
Lundi	30	0.10	20
Mardi	14	0.10	20
Mercredi	34	0.15	30
Jeudi	45	0.20	40
Vendredi	57	0.30	60
Samedi	20	0.15	30

```r
# Vérification conditions
cat("\nCondition n*pi*(1-pi) >= 5 :\n")
```

```
##
## Condition n*pi*(1-pi) >= 5 :
```

```r
for (i in 1:k) {
  val <- n * prob_theo[i] * (1 - prob_theo[i])
  cat(sprintf("  %s : %.2f %s\n", jours[i], val, ifelse(val >= 5, "OK", "NON")))
}
```

```
##   Lundi : 18.00 OK
##   Mardi : 18.00 OK
##   Mercredi : 25.50 OK
##   Jeudi : 32.00 OK
##   Vendredi : 42.00 OK
##   Samedi : 25.50 OK
```

```r
# Statistique Q (sans fonction prédéfinie)
termes <- (observes - theoriques)^2 / theoriques
Q <- sum(termes)
cat(sprintf("\nQ = %.4f\n", Q))
```

```
##
## Q = 11.4417
```

```r
ddl <- k - 1
alpha <- 0.05
chi2_crit <- qchisq(1 - alpha, ddl)
p_val <- 1 - pchisq(Q, ddl)

cat(sprintf("ddl = %d,  ²_critique(5%%) = %.4f\n", ddl, chi2_crit))
```

```
## ddl = 5,  ²_critique(5%) = 11.0705
```

```r
cat(sprintf("p-value = %.6f\n", p_val))
```

```
## p-value = 0.043293
```

```r
if (Q > chi2_crit) {
  cat("Q >  ²_crit => REJET de H0\n")
} else {
  cat("Q <=  ²_crit => On NE REJETTE PAS H0\n")
  cat("La distribution théorique est compatible avec les observations.\n")
  cat("On peut envisager d'ouvrir le restaurant.\n")
}
```

```
## Q >  ²_crit => REJET de H0
```

```r
cat("\nVérification avec chisq.test :\n")
```

```
##
## Vérification avec chisq.test :
```

```r
print(chisq.test(x = observes, p = prob_theo))
```

```
##
##  Chi-squared test for given probabilities
##
## data:  observes
## X-squared = 11.442, df = 5, p-value = 0.04329
```

7 Exercice 7 — AFC : Cas 1

7.1 Partie 1 : Introduction

```r
donnees2_raw <- read.table("donnees2.txt", header = TRUE, row.names = 1)
donnees2 <- donnees2_raw[-nrow(donnees2_raw), -ncol(donnees2_raw)]
cat("Tableau de contingence :\n"); print(donnees2)
```

```
## Tableau de contingence :

##      seri gene gai honn intl serv cour comp disc
## PAYS   10    3  15   26    1   18   13    4   10
## OUVR   18    4  20   41    2   30   18    7   13
## VEND   15    5  13   22    6   15   12    8   10
## COMM    7    6  14   18    6   10    8    8    7
## EMPL   13    7  17   30    5   20   14   10   12
## TECH    5    5   8   11    4    8    5    6    5
## UNIV    2   12   4    8    7    3    2    8    3
## LIBE    3    9   5    7    6    4    3    7    4
```

7.1.1 Question 1 : Pourcentages

```r
total_votes <- sum(donnees2)
cat("Total votes :", total_votes, "\n\n")
```

```
## Total votes : 725
```

```r
pct_qual <- prop.table(colSums(donnees2)) * 100
cat("% par qualité :\n"); print(round(pct_qual, 2))
```

```
## % par qualité :

##  seri  gene   gai  honn  intl  serv  cour  comp  disc
## 10.07  7.03 13.24 22.48  5.10 14.90 10.34  8.00  8.83
```

```r
pct_prof <- prop.table(rowSums(donnees2)) * 100
cat("\n% par profession :\n"); print(round(pct_prof, 2))
```

```
##
## % par profession :

##  PAYS  OUVR  VEND  COMM  EMPL  TECH  UNIV  LIBE
## 13.79 21.10 14.62 11.59 17.66  7.86  6.76  6.62
```

7.1.2 Question 2 : Sondés et proportions

```r
cat("Nombre de sondés :", total_votes / 3, "(3 choix par personne)\n\n")
```

```
## Nombre de sondés : 241.6667 (3 choix par personne)
```

```r
prop1 <- donnees2["EMPL", "honn"] / total_votes
cat("Proportion EMPL×honn / total :", round(prop1*100, 2), "%\n")
```

```
## Proportion EMPL×honn / total : 4.14 %
```

```r
prop2 <- donnees2["EMPL", "honn"] / sum(donnees2[, "honn"])
cat("Proportion EMPL parmi honn :", round(prop2*100, 2), "%\n")
```

```
## Proportion EMPL parmi honn : 18.4 %
```

7.2 Partie 2 : AFC

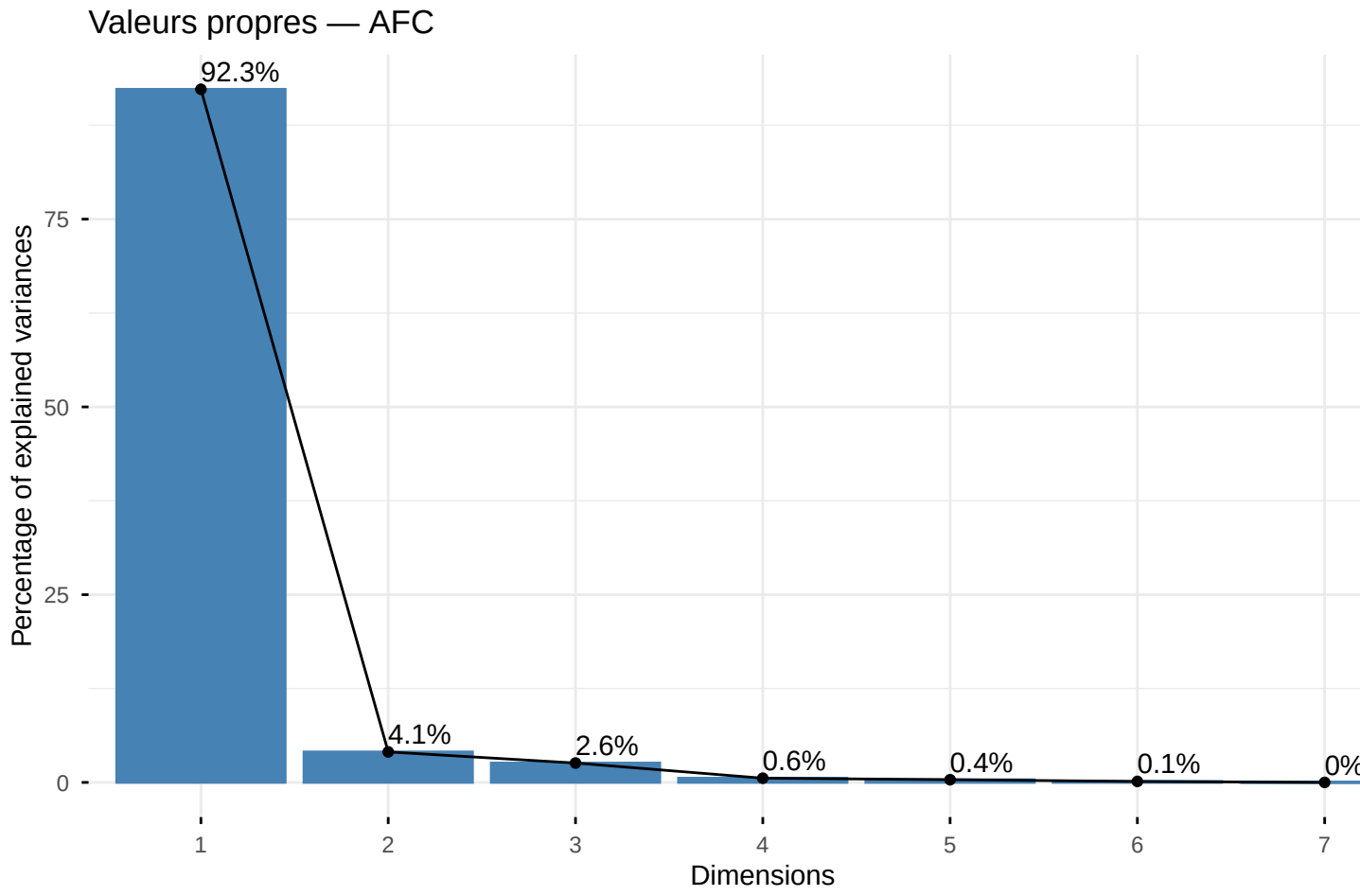
```r
library(FactoMineR); library(factoextra)
res.ca <- CA(donnees2, graph = FALSE)

cat("Valeurs propres :\n")
```

```
## Valeurs propres :
```

```r
print(round(res.ca$eig, 4))
```

```
##       eigenvalue percentage of variance cumulative percentage of variance
## dim 1     0.1232                92.2907                          92.2907
## dim 2     0.0054                 4.0629                          96.3537
## dim 3     0.0034                 2.5835                          98.9371
## dim 4     0.0008                 0.5639                          99.5011
## dim 5     0.0005                 0.3634                          99.8644
## dim 6     0.0002                 0.1323                          99.9968
## dim 7     0.0000                 0.0032                         100.0000
```

```r
fviz_eig(res.ca, addlabels = TRUE, main = "Valeurs propres - AFC")
```



7.2.1 Question 3 : Pourquoi 7 valeurs propres ?

```
cat("min(I-1, J-1) = min(", nrow(donnees2)-1, ",", ncol(donnees2)-1, ") = 7\n")
```

```
## min(I-1, J-1) = min( 7 , 8 ) = 7
```

7.2.2 Question 4 : Modalités définissant les axes

```
cat("Contributions lignes :\n")
```

```
## Contributions lignes :
```

```
knitr::kable(round(res.ca$row$contrib[, 1:2], 2), col.names = c("Axe 1 %", "Axe 2 %"))
```

	Axe 1 %	Axe 2 %
PAYS	10.08	11.95

	Axe 1 %	Axe 2 %
OUVR	15.87	13.84
VEND	0.52	49.17
COMM	0.93	8.59
EMPL	0.96	0.04
TECH	1.47	1.66
UNIV	46.08	14.73
LIBE	24.09	0.02

```r
cat("\nContributions colonnes :\n")
```

```
##
## Contributions colonnes :
```

```r
knitr::kable(round(res.ca$col$contrib[, 1:2], 2), col.names = c("Axe 1 %", "Axe 2 %"))
```

	Axe 1 %	Axe 2 %
seri	3.42	25.33
gene	42.05	24.08
gai	0.98	2.31
honn	4.23	13.78
intl	24.19	18.71
serv	7.63	9.78
cour	4.25	0.37
comp	12.83	4.00
disc	0.42	1.64

```r
cat("\nAxe 1 : LIBE, UNIV (+) vs OUVR, PAYS (-) ; gene vs seri\n")
```

```
##
## Axe 1 : LIBE, UNIV (+) vs OUVR, PAYS (-) ; gene vs seri
```

```r
cat("Axe 2 : PAYS ; gene, disc\n")
```

```
## Axe 2 : PAYS ; gene, disc
```

7.2.3 Question 5 : Modalités mal représentées

```r
cos2_r <- res.ca$row$cos2[, 1] + res.ca$row$cos2[, 2]
cos2_c <- res.ca$col$cos2[, 1] + res.ca$col$cos2[, 2]
cat("Lignes mal représentées :", paste(names(cos2_r[cos2_r < 0.5]), collapse=", "), "\n")
```

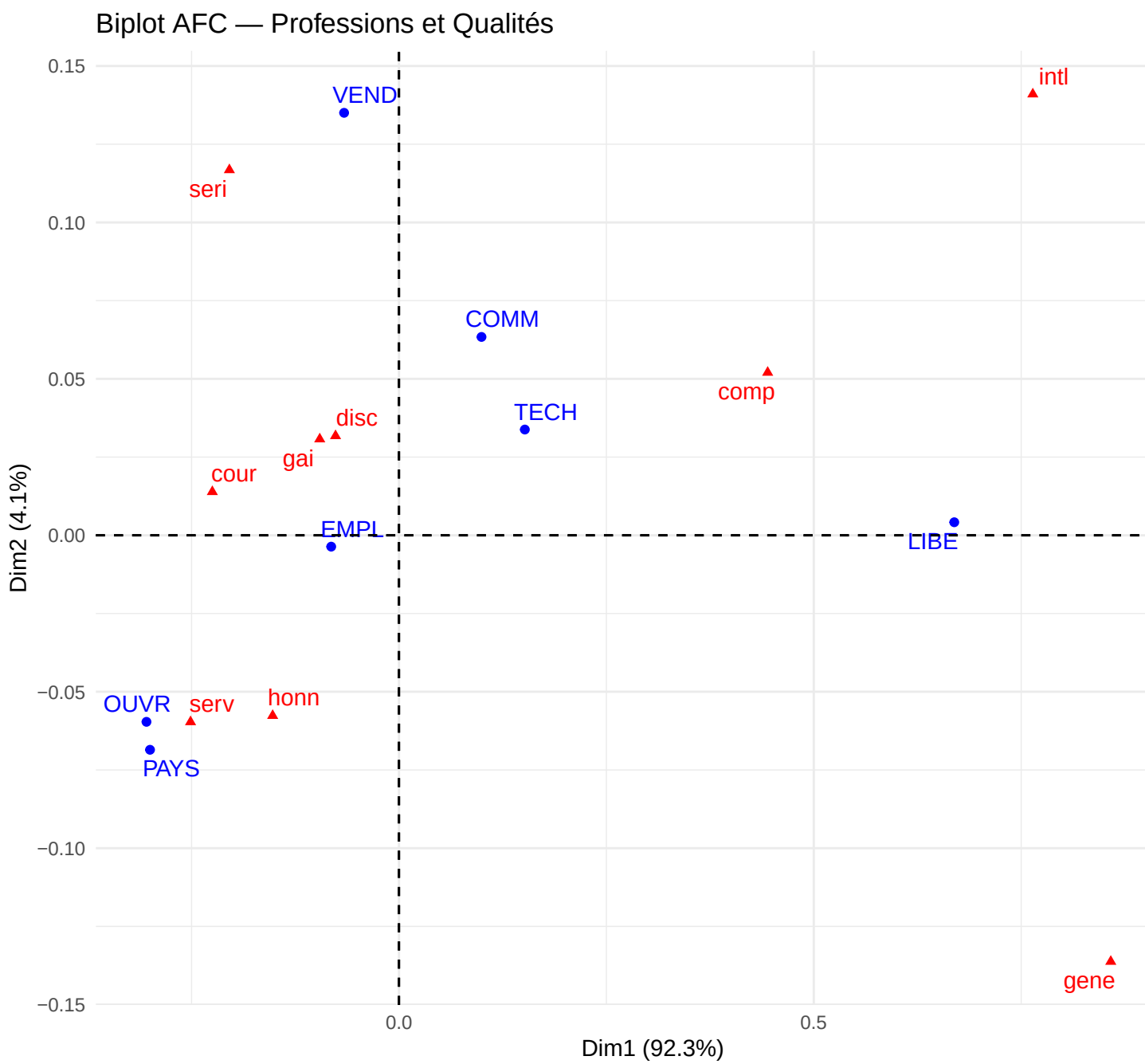
```
## Lignes mal représentées : COMM
```

```r
cat("Colonnes mal représentées :", paste(names(cos2_c[cos2_c < 0.5]), collapse=", "), "\n")
```

```
## Colonnes mal représentées : gai
```

7.2.4 Question 6 : Proximités

```r
fviz_ca_biplot(res.ca, repel = TRUE, col.row = "blue", col.col = "red",
               title = "Biplot AFC - Professions et Qualités") + theme_minimal()
```



```r
cat("OUVR et PAYS proches => profils similaires (mêmes qualités valorisées)\n")
```

```
## OUVR et PAYS proches => profils similaires (mêmes qualités valorisées)
```

```r
cat("VEND et honn proches => vendeurs valorisent particulièrement l'honnêteté\n")
```

```
## VEND et honn proches => vendeurs valorisent particulièrement l'honnêteté
```

8 Exercice 8 — AFC : Élection présidentielle 2007

```r
data_elec <- read.csv("election 2007.csv", sep = ";", row.names = 1)
premier_tour <- data_elec[rownames(data_elec) != "France", ]
noms_reg <- rownames(premier_tour)
premier_tour <- as.data.frame(lapply(premier_tour, as.numeric))
rownames(premier_tour) <- noms_reg

cat(nrow(premier_tour), "régions ×", ncol(premier_tour), "candidats\n")
```

```
## 22 régions × 12 candidats
```

```r
knitr::kable(round(premier_tour, 2), caption = "Résultats 1er tour 2007")
```

Table 11: Résultats 1er tour 2007

	Besancenot	Buffet	Schivardi	Laguiller	Bove	Voynet	Royal	Bayrou	Sarkozy	Nihous	Villiers	Le_Pen
Alsace	3.11	1.39	0.26	1.05	1.04	1.01	21.42	17.56	31.18	1.15	2.34	18.49
Aquitaine	4.68	2.13	0.42	1.70	1.52	1.71	28.79	20.13	26.77	2.07	1.36	8.72
Auvergne	4.43	2.07	0.47	1.88	1.24	1.39	27.11	19.69	28.18	2.14	1.53	9.87
Basse-Normandie	3.95	1.68	0.39	1.47	1.19	1.51	26.72	20.26	29.40	1.86	1.82	9.75
Bourgogne	4.18	2.03	0.45	1.73	1.27	1.46	26.52	19.31	29.14	1.80	1.77	10.34
Bretagne	4.13	1.87	0.35	1.38	1.31	1.88	29.91	22.18	26.27	1.51	1.27	7.94
Centre	4.11	1.79	0.43	1.61	1.21	1.52	27.17	20.08	28.62	1.76	1.63	10.07
Champagne-Ardenne	3.63	1.75	0.40	1.56	1.07	1.23	25.22	18.24	30.66	1.77	2.02	12.45
Corse	2.14	1.23	0.24	0.85	1.57	0.87	23.54	14.88	29.13	0.99	2.49	22.07
Franche-Comte	4.61	2.15	0.51	1.86	1.32	1.42	27.15	18.84	28.85	1.74	1.58	9.97
Haute-Normandie	4.32	2.07	0.45	1.74	1.16	1.43	27.38	18.44	28.93	1.52	1.66	10.90
Ile-de-France	4.13	2.03	0.27	1.27	1.28	1.87	27.85	21.51	28.27	0.67	1.23	9.62
Languedoc-Roussillon	4.86	2.17	0.51	1.77	1.51	1.53	28.38	17.99	27.18	1.56	1.46	11.08
Limousin	5.46	2.74	0.54	2.17	1.41	1.59	30.70	19.27	25.41	2.26	1.35	7.10
Lorraine	3.57	1.82	0.37	1.44	1.07	1.22	25.13	18.64	30.33	1.43	2.13	12.85
Midi-Pyrenees	4.82	2.15	0.49	1.72	1.52	1.78	30.16	20.64	25.73	1.82	1.23	7.94
Nord-Pas-de-Calais	4.40	2.65	0.45	2.01	1.16	1.19	28.63	16.49	27.13	1.27	1.33	13.29
Pays-de-la-Loire	3.56	1.50	0.35	1.27	1.23	1.72	27.89	23.30	28.37	1.66	1.62	7.53
Picardie	3.86	1.93	0.46	1.73	1.12	1.23	25.81	17.78	29.54	1.75	1.89	12.90
Poitou-Charentes	4.39	1.89	0.45	1.68	1.34	1.62	31.03	20.57	26.18	2.02	1.36	7.47
Provence-Alpes-Cote-d-Azur	3.51	1.56	0.32	1.20	1.30	1.28	24.12	18.07	29.95	0.93	2.25	15.51
Rhone-Alpes	4.04	1.73	0.33	1.36	1.31	1.68	26.93	21.27	29.06	1.25	1.44	9.60

8.1 Test du Chi-deux

```
chi2 <- chisq.test(premier_tour)
cat(" ² =", round(chi2$statistic, 2), ", ddl =", chi2$parameter,
    ", p =", format(chi2$p.value, scientific=TRUE), "\n")
```

```
##  ² = 46.81 , ddl = 231 , p = 1e+00
```

```
cat("=> Rejet de H0 : lien significatif régions/candidats => AFC justifiée\n")
```

```
## => Rejet de H0 : lien significatif régions/candidats => AFC justifiée
```

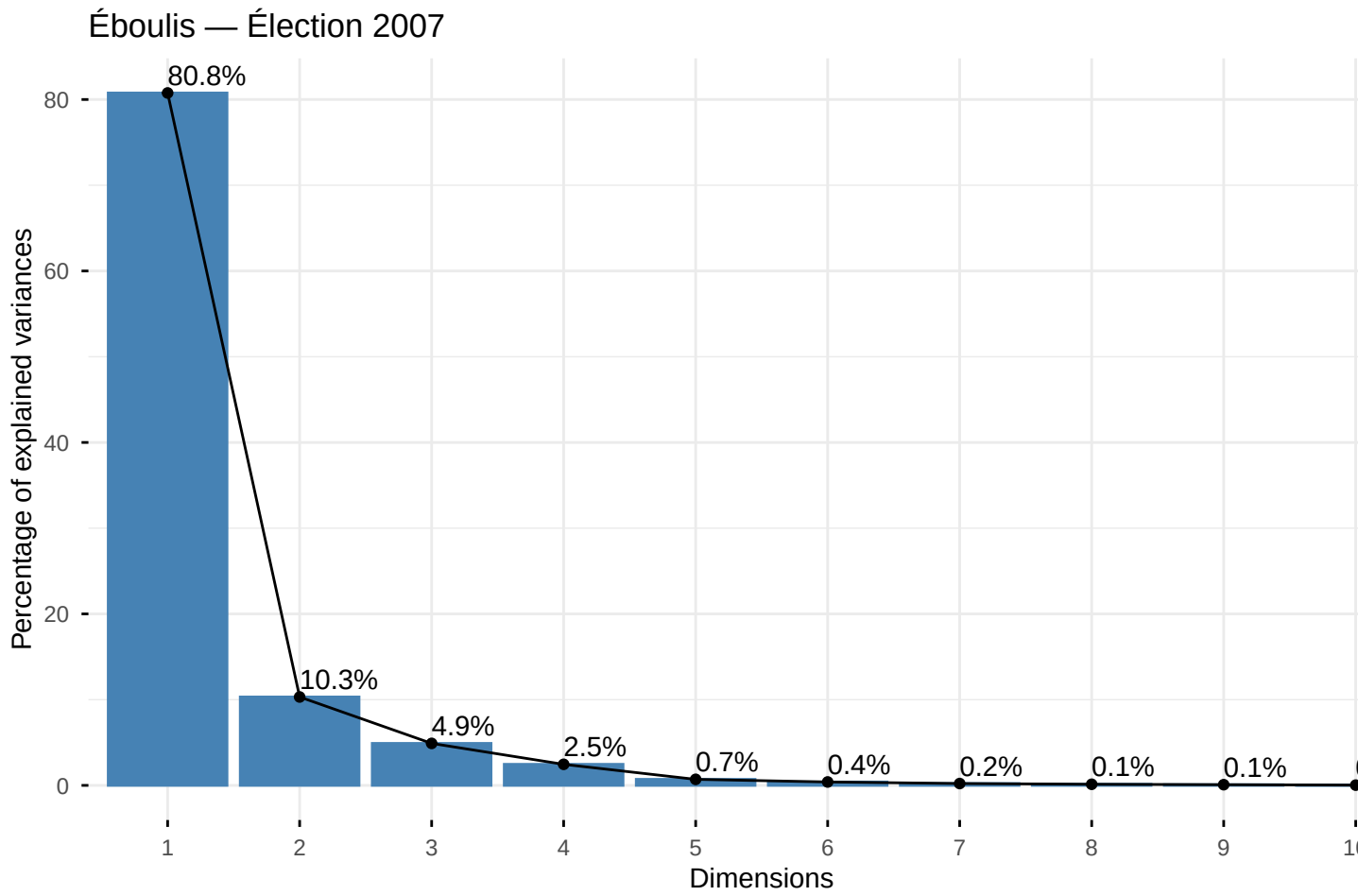
8.2 AFC

```
afc_e <- CA(premier_tour, graph = FALSE)
knitr::kable(round(afc_e$eig, 2),
             col.names = c("VP", "% inertie", "% cumulé"),
             caption = "Valeurs propres - Élection 2007")
```

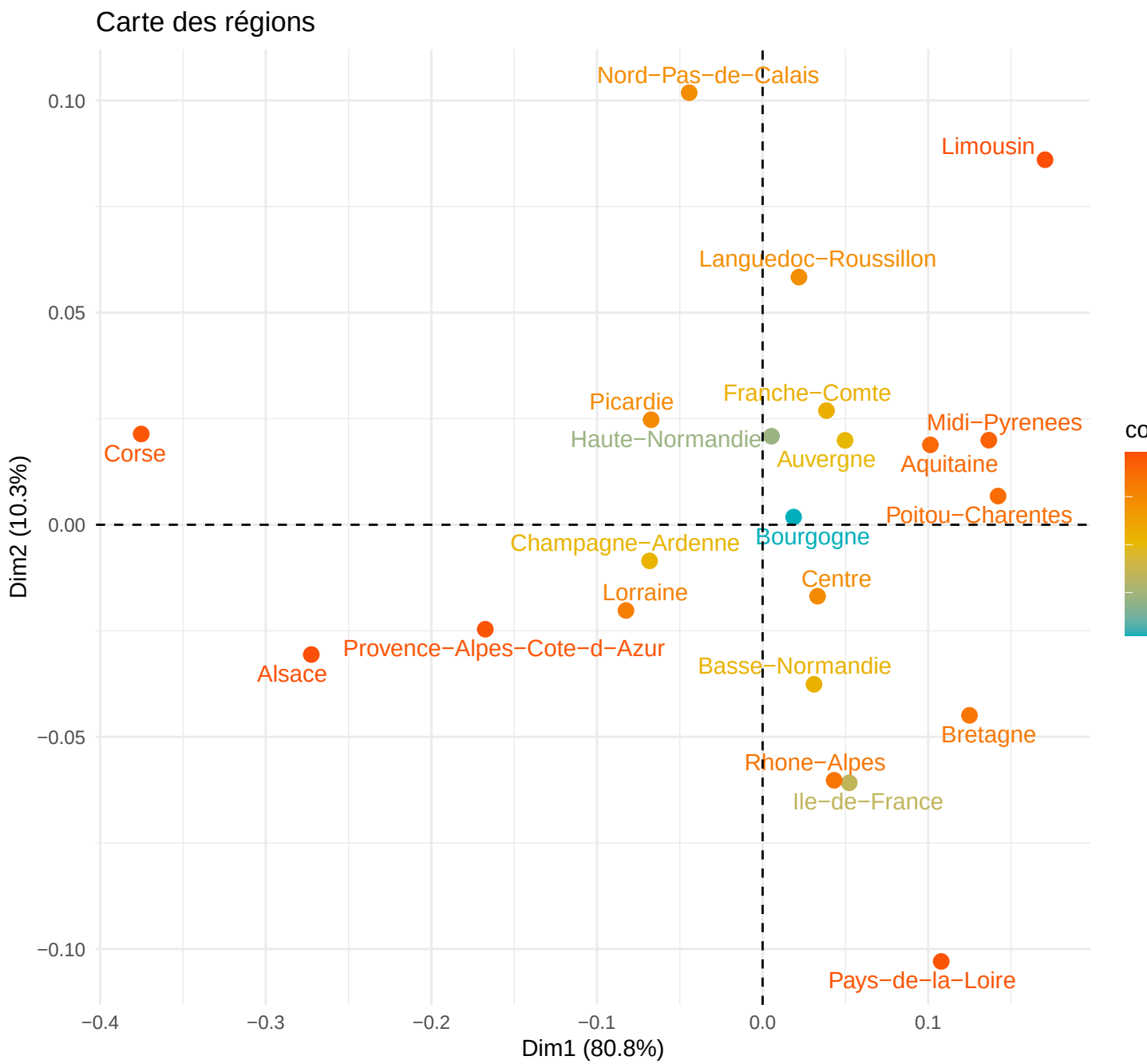
Table 12: Valeurs propres — Élection 2007

	VP	% inertie	% cumulé
dim 1	0.02	80.76	80.76
dim 2	0.00	10.31	91.08
dim 3	0.00	4.90	95.97
dim 4	0.00	2.46	98.43
dim 5	0.00	0.71	99.13
dim 6	0.00	0.40	99.53
dim 7	0.00	0.22	99.75
dim 8	0.00	0.13	99.88
dim 9	0.00	0.08	99.96
dim 10	0.00	0.03	99.99
dim 11	0.00	0.01	100.00

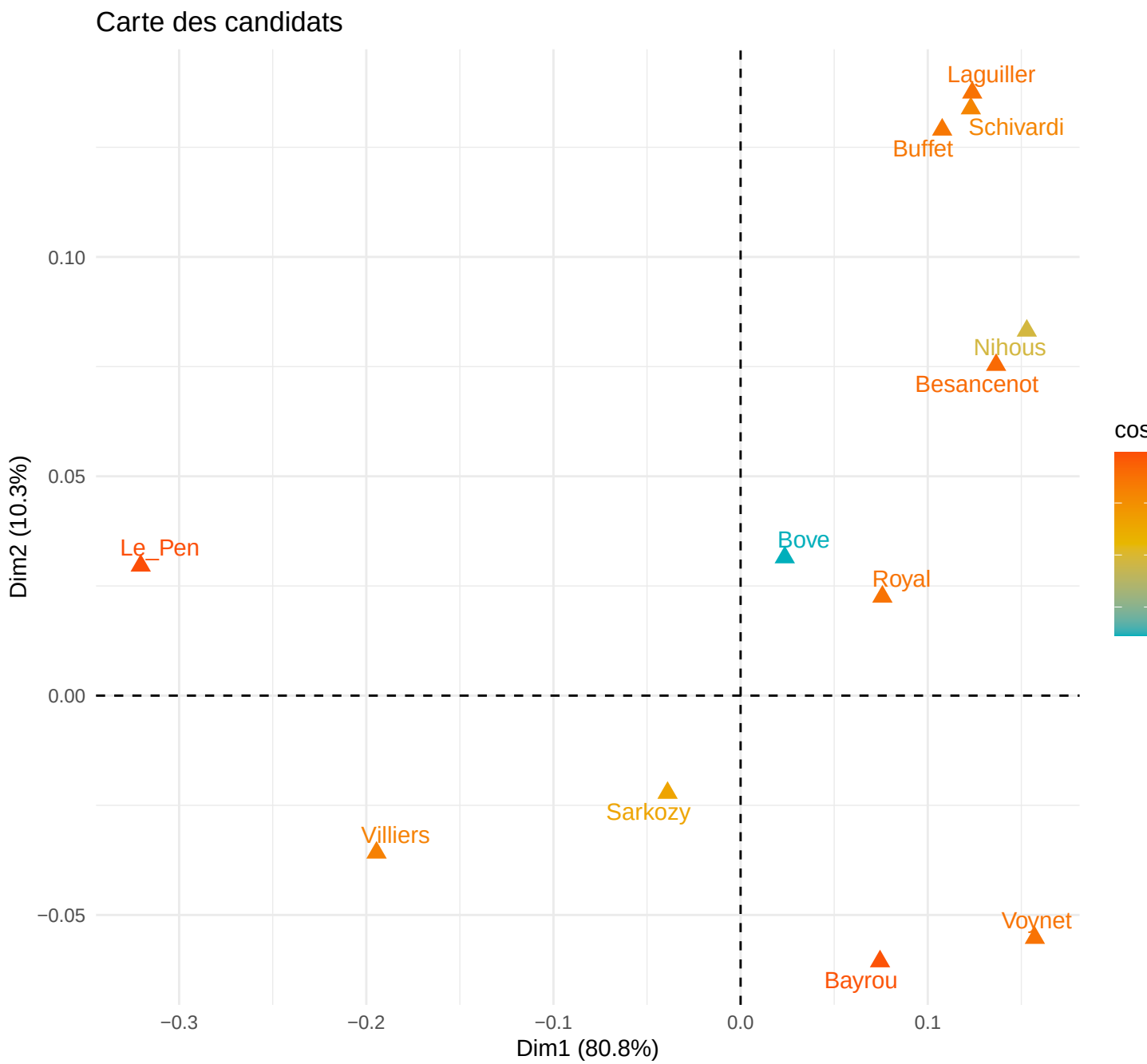
```
fviz_eig(afc_e, addlabels = TRUE, main = "Éboulis - Élection 2007")
```



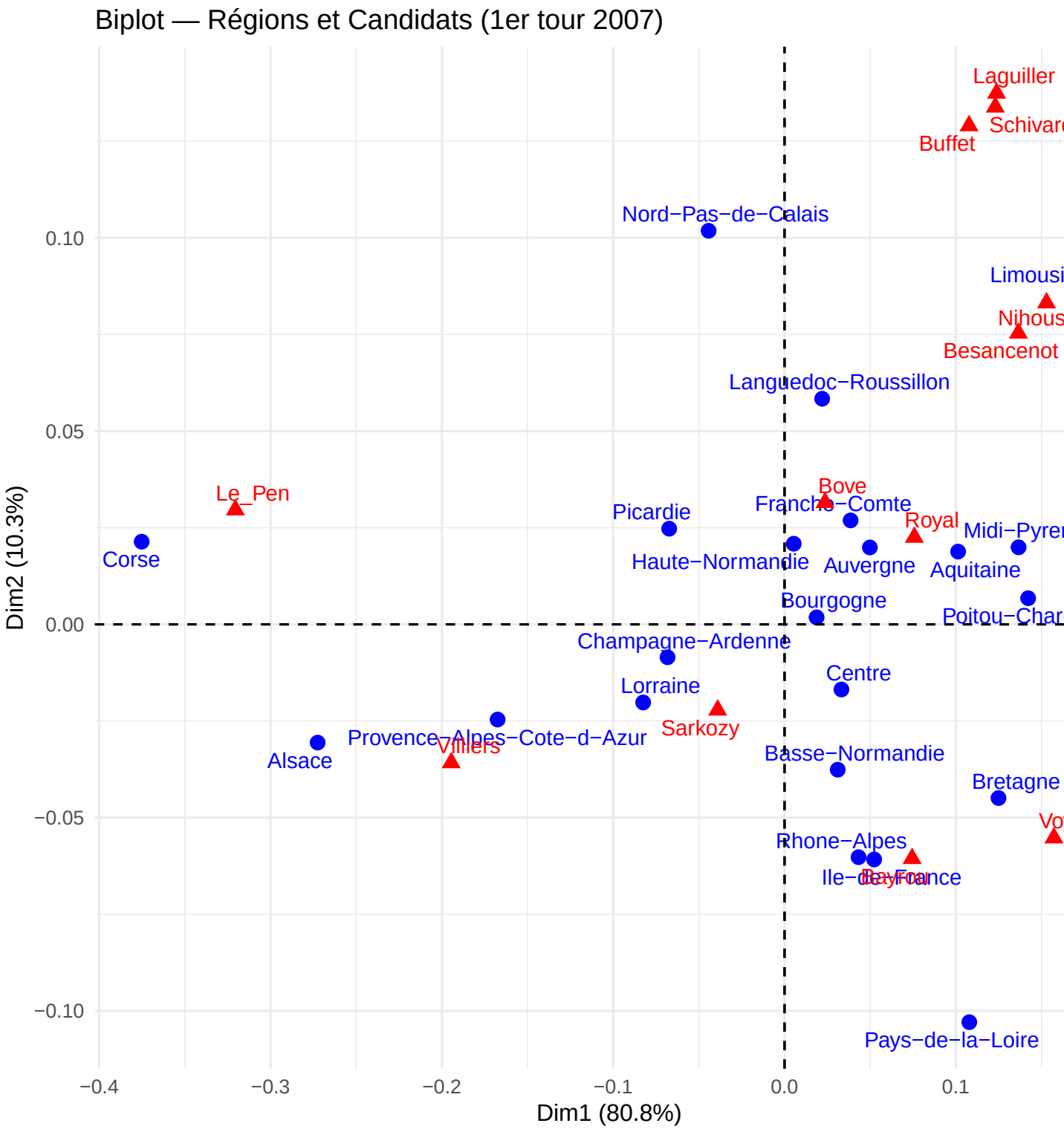

```
fviz_ca_row(afc_e, col.row = "cos2",
            gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
            repel = TRUE, pointsize = 3,
            title = "Carte des régions") + theme_minimal()
```




```r
fviz_ca_col(afc_e, col.col = "cos2",
            gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
            repel = TRUE, pointsize = 3,
            title = "Carte des candidats") + theme_minimal()
```




```
fviz_ca_biplot(afc_e, col.row = "blue", col.col = "red",
               repel = TRUE, pointsize = 3,
               title = "Biplot - Régions et Candidats (1er tour 2007)") +
  theme_minimal(base_size = 12)
```




```r
cat("Contributions régions :\n")
```

```
## Contributions régions :
```

```r
knitr::kable(round(afc_e$row$contrib[, 1:2], 2), col.names = c("Dim1 %", "Dim2 %"))
```

	Dim1 %	Dim2 %
Alsace	19.64	1.94
Aquitaine	2.71	0.73
Auvergne	0.66	0.82
Basse-Normandie	0.26	2.93
Bourgogne	0.09	0.01
Bretagne	4.13	4.18
Centre	0.29	0.59
Champagne-Ardenne	1.23	0.15
Corse	37.24	0.95
Franche-Comte	0.39	1.50
Haute-Normandie	0.01	0.90
Ile-de-France	0.72	7.67
Languedoc-Roussillon	0.13	7.06
Limousin	7.69	15.33
Lorraine	1.80	0.85
Midi-Pyrenees	4.94	0.82
Nord-Pas-de-Calais	0.52	21.48
Pays-de-la-Loire	3.08	21.95
Picardie	1.20	1.27
Poitou-Charentes	5.35	0.09
Provence-Alpes-Cote-d-Azur	7.42	1.26
Rhone-Alpes	0.49	7.52

```r
cat("\nContributions candidats :\n")
```

```
##
## Contributions candidats :
```

```r
knitr::kable(round(afc_e$col$contrib[, 1:2], 2), col.names = c("Dim1 %", "Dim2 %"))
```

	Dim1 %	Dim2 %
Besancenot	4.43	10.58
Buffet	1.30	14.59
Schivardi	0.36	3.31
Laguiller	1.40	13.48
Bove	0.04	0.58
Voynet	2.10	2.03
Royal	9.07	6.29
Bayrou	6.24	32.33
Sarkozy	2.51	6.36
Nihous	2.16	5.01
Villiers	3.68	0.98
Le_Pen	66.71	4.46

```r
cat("\nDim 1 : Le Pen, Laguiller, Nihous (droite) vs Royal, Bayrou, Voynet (gauche)\n")
```

```
##
## Dim 1 : Le Pen, Laguiller, Nihous (droite) vs Royal, Bayrou, Voynet (gauche)
```

```r
cat("  Régions : Corse, PACA, Alsace vs Bretagne, Pays-de-la-Loire, Midi-Pyrénées\n")
```

```
##   Régions : Corse, PACA, Alsace vs Bretagne, Pays-de-la-Loire, Midi-Pyrénées
```

```r
cat("\nDim 2 : Nihous, Besancenot (haut) vs Sarkozy, Le Pen (bas)\n")
```

```
##
## Dim 2 : Nihous, Besancenot (haut) vs Sarkozy, Le Pen (bas)
```

```r
cat("  Régions : Limousin, Poitou-Charentes vs Alsace, PACA, Corse\n")
```

```
##   Régions : Limousin, Poitou-Charentes vs Alsace, PACA, Corse
```

```r
cat("\nConclusion : structuration géographique claire des préférences électorales.\n")
```

```
##
## Conclusion : structuration géographique claire des préférences électorales.
```